

UNIVERSIDAD POLITÉCNICA DE
MADRID

ESCUELA TÉCNICA SUPERIOR DE
INGENIEROS DE TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA
DE SISTEMAS ELECTRÓNICOS

MASTER'S THESIS

DESIGN OF A LOW-POWER FFT
ARCHITECTURE FOR 6G

MANUEL VIQUEIRA RODRÍGUEZ

JULY 2021

MÁSTER UNIVERSITARIO EN INGENIERÍA DE SISTEMAS ELECTRÓNICOS

MASTER'S THESIS

Title: DESIGN OF A LOW-POWER FFT ARCHITECTURE FOR 6G

Author: MANUEL VIQUEIRA RODRÍGUEZ

Supervisor1: MARIO GARRIDO

Supervisor2: MARÍA LUISA LOPEZ VALLEJO

Department: DEPARTAMENTO DE INGENIERÍA ELECTRÓNICA

EVALUATION COMMITTEE

President:

Vocal:

Secretary:

The evaluation committee members agree to grant the grade:

Madrid, on, 2021

UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE
TELECOMUNICACIÓN



MÁSTER UNIVERSITARIO EN INGENIERÍA DE
SISTEMAS ELECTRÓNICOS

MASTER'S THESIS

DESIGN OF A LOW-POWER FFT ARCHITECTURE FOR 6G

AUTHOR: MANUEL VIQUEIRA RODRÍGUEZ

SUPERVISOR 1: MARIO GARRIDO

SUPERVISOR 2: MARÍA LUISA LOPEZ VALLEJO

JULY 2021

*A mi familia y amigos por todo el apoyo y dedicación en estos duros tiempos.
Gracias también a mis supervisores Mario y Marisa por todo el apoyo en la
elaboración de esta tesis y la flexibilidad durante todo el año.*

"El conocimiento siempre es la más segura inversión, especialmente en estos tiempos", Me.

Abstract

The objective of this Master's Thesis work is the design of a low-power architecture for the Fast Fourier Transform (FFT) with application-specific integrated circuit (ASIC) technology for its use in 6G. The interest in this field comes up from the new requirements of low power in the 6G technology, which oscillates around 1 Tb/J of energy efficient in the complete system. The FFT is a key element in the physical layer of communication systems. Following this, any kind of reduction in the power consumption of the FFT have a direct impact in the power consumption of the system as a whole.

For this purpose, firstly a research about the state-of-the-art of the different components of an FFT has being carried out, and the possible techniques available for its design have being analyzed.

Once the state-of-the-art was analyzed, a series of possible improvements to these components have been proposed, with the objective of reducing as much as possible the power consumption of the FFT as a whole.

After that, all architectures have been implemented using VHDL. In parallel, some components that required synthesis tools that we did not have were ordered to the consortium Europractice. Once the implementation and testing of the different components was over, the full FFTs architectures were implemented and tested.

After finishing all VHDL implementations, all different components and FFTs have been synthesized into TSMC 40 nm standard cells using Design Compiler synthesizer.

Finally, all proposed designs have been analyzed and compared for achieving an FFT architecture with the lowest possible power consumption. Experimental results show that the proposed FFT architecture achieves a power consumption of only 7.73 mW.

Key Words

FFT, VHDL, ASIC, radix, shift-and-add, CORDIC, algorithmic transformations.

Resumen

El objetivo de este Trabajo de Fin de Máster es el diseño de una arquitectura de la transformada rápida de Fourier (FFT) de bajo consumo sobre *application-specific integrated circuit* (ASIC) para su uso en 6G. El interés en este campo viene motivado por los nuevos requisitos de bajo consumo de la tecnología 6G, que se cifran en 1 Tb/J de eficiencia energética del sistema completo. La FFT es un elemento fundamental de la capa física de los sistemas de comunicaciones. Así, cualquier reducción en el consumo de potencia tendrá un impacto directo en el consumo del sistema.

Para este fin, primeramente se ha realizado una investigación del estado del arte de los componentes de una FFT, y se han analizando las técnicas disponibles para su diseño.

Una vez analizado el estado del arte, se han propuesto una serie de posibles mejoras a estos componentes, con el objetivo de reducir lo máximo posible el consumo de potencia de la FFT en su conjunto.

Tras esto, se han implementado las arquitecturas a analizar usando VHDL. En paralelo, se han solicitado algunos componentes que requerían sintetizadores con los que no contábamos al consorcio Europractice. Una vez acabada la implementación y el testeo de todos los componentes, se han implementado y testado las arquitecturas FFTs al completo.

Al terminar todas las implementaciones en VHDL, se ha procedido a la síntesis de todos los componentes y FFTs en células estándar para tecnología TSMC de 40 nm por medio del sintetizador Design Compiler.

Finalmente, todos los diseños propuestos han sido analizados y comparados para conseguir la arquitectura FFT con el menor consumo de potencia posible. Los resultados experimentales muestran que la arquitectura propuesta consume sólo 7.73 mW.

Palabras clave

FFT, VHDL, ASIC, radix, shift-and-add, CORDIC, transformaciones algorítmicas.

Acknowledgments

Thanks to my family and friends for all the support during this hard times. Thanks as well to my supervisors Marisa and Mario for the great support during for the elaboration of this Thesis and for the remote flexibility during the whole year.

Contents

1	Introduction	1
1.1	Mobile telecommunication, past and present	1
1.2	6G	2
1.3	Aim of the Thesis	3
1.4	Design Methodology	3
2	Background	5
2.1	FFT algorithms	5
2.1.1	Representation	6
2.2	FFT architectures	8
2.2.1	SDF FFT architecture	9
2.2.2	SFF FFT architecture	10
2.2.3	Memory or shift-register based butterflies	10
2.3	Rotators	12
2.3.1	Constant rotators	12
2.3.2	General rotators	14
3	Hardware implementation	17
3.1	Proposed designs	17

3.1.1	Memory implementation	18
3.1.2	Alternative SFF FFT architecture	19
3.1.3	Transformations to simplify rotators	21
3.2	Rotators distribution among the FFT, Radix-2 study	22
3.3	FFT control	23
3.4	Rotators control	24
3.5	Trivial rotators implementation	27
3.6	Multiplier-based implementations	27
3.6.1	The W_8 rotator implemented using multipliers	28
3.6.2	The W_{32} rotator implemented using multipliers	30
3.7	CCSSI rotators	31
3.7.1	The W_8 rotator implemented using shift-and-add	31
3.7.2	The W_{32} rotator implemented using shift-and-add	32
4	Experimental results and comparison	39
4.1	FFT stages comparison	39
4.2	Rotators comparison	42
4.3	Full FFTs comparison	43
4.4	Comparison with previous serial FFTs on ASICs	44
5	Conclusion and future work	49
5.1	Conclusion	49
5.2	Future work	50
A	Budget	51
B	Ethical, social, economic and environmental aspects	53

B.1	Ethical and social impacts	53
B.2	Economic impact	54
B.3	Environmental impact	54

List of Figures

2.1	Flow graph of a 16-point Radix-2 DIF FFT.	7
2.2	16 point Radix-2 DIF FFT representations. (a) Binary tree and rotation distribution among stages. (b) Triangular matrix.	8
2.3	Stage of an SDF FFT architecture.	9
2.4	Stage of an SFF FFT architecture.	10
2.5	$L = 8$ memory data-flow, only impacts one address.	11
2.6	$L = 8$ Shift Register data-flow, data moves all over the shift register.	11
2.7	W_4 , W_8 and W_{16} twiddle factors.	12
2.8	Direct multiplication strategy architecture.	13
2.9	CORDIC algorithm examples. (a) Traditional CORDIC algorithm. (b) Redundant CORDIC algorithm.	15
3.1	Proposed L delay architecture with 2 $L/2$ RAM memories working in parallel.	18
3.2	SFF Architecture improvement for reducing memory accesses implemented with memory registers.	20
3.3	SFF Architecture improvement for reducing memory accesses implemented with RAM delays.	21
3.4	Decimation of a rotator. Conversion of a W_{16} into a W_8 + a 22.5° angle rotation.	22
3.5	Radix- 2^4 approach.	23
3.6	Radix- 2^5 approach.	24

3.7	Distribution of the rotators among the FFT.	24
3.8	10-bits counter in charge of the control of the FFT components. . . .	25
3.9	1024-inputs Radix-2 ⁵ FFT triangular matrix.	27
3.10	Twiddle factor n ^o 5 control bits from the triangular matrix.	28
3.11	T_4 Rotator for 0 ^o and -90 ^o rotations.	30
3.12	W_8 ROM and ROM control architecture.	30
3.13	W_8 decimation into a T_4 and a -45 ^o rotation.	33
3.14	W_8 shift-and-add architecture.	34
3.15	W_{32} decimation into a W_8 + a -33.75 ^o /-22.5/-11.25 ^o rotation.	36
3.16	W_{32} modifications following decimation, reduction, center, move and merge transformations [1].	36
3.17	W_8 shift-and-add architecture allowing as well 180 ^o rotations.	37
3.18	$\pm 5.625^\circ / \pm 16.875^\circ$ Rotator architecture.	38
4.1	Area of different FFT stages (without rotator) as a function of the buffer length, L	40
4.2	Power consumption of different FFT stages (without rotator) as a function of the buffer length, L	41

List of Tables

1.1	Generation comparison in data transfer speed and latency.	2
2.1	SDF radix-2 values over time.	9
2.2	SFF table of values over time.	10
3.1	$L = 8$ delay implemented with 2 $L = 4$ RAM memories behavior. . .	19
3.2	Behavior of the memory used for this Thesis.	20
3.3	SFF alternative, table of values over time for $L = 8$	22
3.4	Radix-2 ⁴ and Radix-2 ⁵ rotators required.	25
3.5	Comparison of rotators types in Radix-2 ⁴ and Radix-2 ⁵ approaches. .	25
3.6	Radix-2 ⁵ latency between components.	26
3.7	Rotation angles according to triangular matrix.	29
3.8	T_4 , $\phi_1(I)$ control bits.	29
3.9	rotation $\phi_4(I)$ corresponding with the twiddle factor W_8 , control bits and angles explosion.	29
3.10	$\phi_4(I)$, $\phi_9(I)$ equivalences of control inputs.	31
3.11	W_{32} , $\phi_2(I)$ control bits and angles explosion.	32
3.12	$\phi_2(I)$, $\phi_7(I)$ equivalences of control inputs.	33
3.13	W_8 , $\phi_4(I)$ control bits.	33
3.14	W_{32} , $\phi_2(I)$ control bits for shift-and-add rotator.	35

3.15	W_8 control bits and angles resultant.	36
3.16	$\pm 5.625^\circ / \pm 16.875^\circ$ Rotator control and coefficients.	36
4.1	Comparison of FFT stages for low values of L	45
4.2	Comparison of FFT stages for high values of L	46
4.3	Comparison of rotators.	46
4.4	Expansion of 1024-point FFT architectures.	47
4.5	Comparison of serial FFTs implemented on ASICs.	47
A.1	Budget.	52

Chapter 1

Introduction

The Fourier transform is a mathematical algorithm capable of transforming signals from the time domain into the frequency domain. This transform is widely used in communications as it allows the modulation/demodulation of the symbols in the physical layer [2]. However, current hardware architecture techniques are far from achieving the required performance for 6G communications and, as result, new techniques will be required to develop 6G communications in the upcoming years.

1.1 Mobile telecommunication, past and present

The mobile telecommunications history beginning [3] can be dated in the decade of the 50s, when the first mobile telecommunications devices appear. This first devices represent the so called first generation (0G) or pre-cellular. These devices were called radio telephone and they were extremely heavy, more or less portable and allowed communication in both ways. This kind of devices are called pre-cellular because the communication was not distributed in cells, technology which will come up in the decade of the 80s by the hand of the Nippon Telegraph and Telephone Corporation in Japan causing a revolution in the telecommunications sector and bringing with it the 1G generation of devices. This cellular communication technology is based on the usage of the same transmitter tower for connecting different devices thanks to the distribution of those into different frequencies.

The next step was 2G which involved the first digital communications. After this step, the data transfer speed and latency of digital communications improved significantly every 10 years with each change of generation, allowing the usage of internet in every device. This performance increase in terms of data transfer speed and latency is shown in table 1.1.

Table 1.1: Generation comparison in data transfer speed and latency.

	1G	2G	3G	4G	5G
Period	1980-1990	1990-2000	1990-2000	2010-present	present-(2030)
Data Bandwidth	2 kb/s	64 kb/s	2 Mb/s	100 Mb/s	10 Gb/s
Latency	>1000 ms	700 ms	200 ms	100 ms	1 ms

On current years, mobile telecommunications are moving from 4G into 5G. This will bring a substantial improvement in data transfer and reduce 100 times the latency. As an example of what this means, a high-definition movie which takes 50 minutes to download in a 4G network will take just 9 when using a 5G one.

In the dawn of this upcoming 5G era, researchers are starting to focus on the next step, the 6G. However, current technologies will require a big upgrade for allowing this next step, expected to come in 2030-2040.

1.2 6G

As seen in section 1.1, the new 5G communications achieve data transfer speeds of up to 10 Gb/s with a latency of 1 ms. This is clearly a great improvement compared to previous 4G technology. However, this performance will not be enough for future applications such as connected autonomous vehicles (CAV) [4] or telesurgery [5] which require ultra-reliable and low latency communications (URLLC) with latency below 100 μ s [6] and data rates of 1 Tb/s.

These performance rates, taking as well under consideration that technology should be sustainable and used in a sustainable way [7], conform the key performance indicators (KPIs) of the future 6G technology. A game changer expected to come in the next 10 or 15 years. Up to this point, it is important to ask to ourselves: “Is technology currently ready for 6G”

So far, the answer is “no”. All components will require further optimization in order to achieve that performance as, if only one of the components exceeded latency values or is not capable of following 1Tb/s data transfer speeds, all the system performance might be compromised.

One of these components is the fast Fourier transformation (FFT) [8] which is an algorithm able of transforming data from time domain to the frequency domain. This transformation is key in mobile telecommunications as it is used for modulation/demodulation of the symbols in the physical layer [2], as well as it can be used for processing spatial signals such as audio, image or video.

The FFT is an efficient way to calculate the discrete Fourier transform (DFT).

The DFT is calculated as

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}nk}, \quad k = 0, 1, \dots, N-1, \quad (1.1)$$

where $x[n]$ is the complex input data from time domain, $X[k]$ is the complex output value in the frequency domain and $e^{-j\frac{2\pi}{N}nk}$ is the so called *twiddle factor* and corresponds to a rotation in the complex plane.

About the KPIs for DFT architectures, a trade-off between the KPIs is always required. This is caused because an increase in the data transfer speed can impact latency, consumption or area. Achieving a good balance between them is a real challenge. For the purpose of this Thesis, we will focus on achieving as least power consumption as possible, while maintaining a reasonable balance with the other KPIs and the area required.

1.3 Aim of the Thesis

As discussed in section 1.2, power consumption will be a key factor in the future 6G technology. For achieving low power consumption, ASIC designs are always more interesting than FPGA ones as the performance difference between them in terms of power consumption is significant. The configurability of FPGAs comes at a price, large capacitances, that result in poor performance and high power dissipation. Taking this in mind, it was decided for this project to use ASIC.

Even when using ASICs, newer architectures for DFTs will be required so, on this project, we will propose new lower power consumption architectures of butterflies and rotators (chapter 3) and we will compare those designs with the ones currently in use (chapter 2).

The main objective of this Thesis will be to obtain a 1024-point-FFT architecture with the lowest power consumption performance and compare it with other similar architectures pursuing best power consumption performance. This architectures will be generated by using Synopsys Design Compiler in 40nm TSMC standard cells.

1.4 Design Methodology

For this project, the design methodology will follow these steps:

1. Design in VHDL various butterflies and rotators. These components are described in chapters 2 and 3.

2. Synthesis by using ModelSim to ensure that they work correctly.
3. Synthesis of these architectures (excluding RAM and ROM memories) using Synopsys Design Compiler. Memories shall be excluded as the 40nm compiler which will be used is proprietary and is executed elsewhere.
4. Describe the memories required for those architectures and send them to Europractice [9] as they have the memory compiler for this technology.
5. Characterization of the different components in terms of area, latency, maximum frequency and power consumption.
6. Build the FFT architectures from the components previously elaborated.
7. Simulate the FFT architectures using ModelSim.
8. Synthesize the FFT architectures and characterize, them in a similar way as done for the independent components.

Chapter 2

Background

As explained in chapter 1, the FFT is a key component in communication. The arithmetic complexity of the equation (1.1) is $O(N^2)$ where N is the number of inputs to the DFT. However, this calculation includes some redundancies that can be optimized by using different FFT algorithms [8, 10].

2.1 FFT algorithms

The Cooley-Tukey algorithm [11] decomposes an N -point DFT into smaller DFTs $N = N_2 \cdot N_1$. By considering that

$$\begin{aligned} n &= n_1 N_2 + n_2, & \text{with } n_1 &= 0, \dots, N_1 - 1, & \text{and } n_2 &= 0, \dots, N_2 - 1; \\ k &= k_2 N_1 + k_1, & \text{with } k_1 &= 0, \dots, N_1 - 1, & \text{and } k_2 &= 0, \dots, N_2 - 1, \end{aligned}$$

the DFT equation 1.1 can be written as

$$X[k_2 N_1 + k_1] = \sum_{n_2=0}^{N_2-1} \sum_{n_1=0}^{N_1-1} x[n_1 N_2 + n_2] \cdot W_N^{(n_1 N_2 + n_2)(k_2 N_1 + k_1)}. \quad (2.1)$$

Finally, by exploiting the periodicity of the twiddle factors, i.e., $W_N^{\phi N_2} = W_{N_1}^{\phi}$,

$W_N^{\phi N_1} = W_{N_2}^{\phi}$ and $W_N^{\phi N_1 N_2} = 1$, the equation is transformed into

$$X[k_2 N_1 + k_1] = \underbrace{\sum_{n_2=0}^{N_2-1} \left[\underbrace{\left(\sum_{n_1=0}^{N_1-1} x[n_1 N_2 + k_1] \cdot W_{N_1}^{n_1 k_1} \right)}_{N_1\text{-point DFT}} \underbrace{W_N^{n_2 k_1}}_{\text{Twiddle factor}} \right]}_{N_2\text{-point DFT}} W_{N_2}^{n_2 k_2}. \quad (2.2)$$

In general, N can be the product of several numbers $N = N_{m-1} \cdot N_{m-2} \cdot \dots \cdot N_0$. Although they technically could be any value, most of the times they are powers of 2. Radix- r refers to the case $N_i = r, \forall_i = 0 \dots m-1$.

The Cooley-Tukey algorithm can be applied by starting from the time domain, which is called decimation-in-time (DIT) decomposition, or starting from the frequency domain, which is called decimation-in-frequency (DIF) decomposition.

2.1.1 Representation

There are many ways of representing FFT algorithms. In this section, the main representation techniques will be briefly explained using as the main example a 16-point radix-2 decimation-in-frequency (DIF) FFT.

A very clear way of representing a DIF is by using flow graphs as shown in figure 2.1. In it, the numbers at the input represent the indexes of the input sequence $x[n]$, whereas those at the output represent the frequencies k , of the outputs $X[k]$. These flow graphs consist in a series of n stages (in the case of figure 2.1, four stages) that include butterflies and rotators.

The butterflies consist in additions and subtractions in pairs. The upper part of the butterflies outputs the addition and the lower part outputs the subtraction. An in-deep description of how these butterflies are implemented in hardware can be seen in section 2.2 and our proposed low power consumption butterfly is described in chapter 3.

Rotations are represented as the numbers in between the stages and correspond to multiplications in the complex plane as

$$W_N^{\phi} = e^{-j \frac{2\pi}{N} \phi}. \quad (2.3)$$

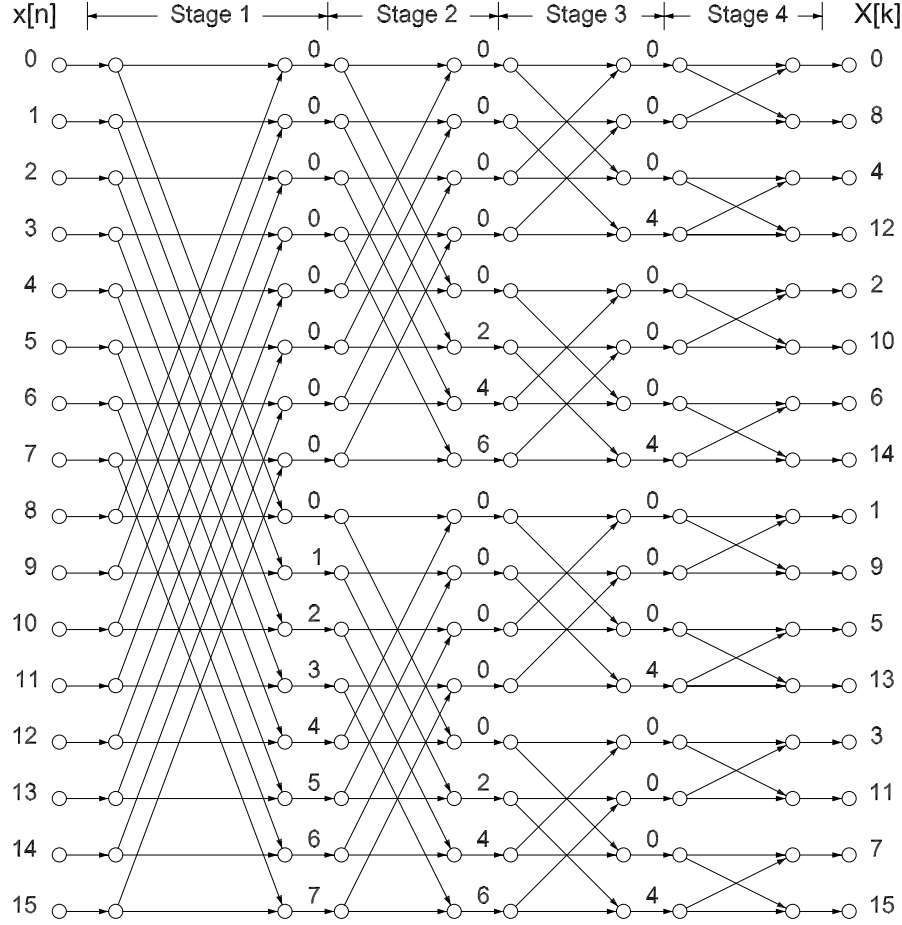


Figure 2.1: Flow graph of a 16-point Radix-2 DIF FFT.

These multiplications in the complex plane can be implemented by using different techniques, which will be explored in section 2.3. Also, our proposed low power consumption rotator techniques are described in chapter 3.

Apart from the flow graph representation, the other two main representations are the binary tree and the triangular matrix. These two representations will help us with the distribution of the different rotators among the FFT stages.

The binary tree representation [12, 13] is shown in figure 2.2(a). It is a generalization of the Cooley-Tukey algorithm in which the N factor decomposition $N = N_{m-1} \cdot N_{m-2} \cdot \dots \cdot N_0$ is represented. Note that, as we are using a DIF for the example, the highest division which corresponds to the longest twiddle factors are located at the earliest stages. In case the distribution were DIT, the largest rotators would be located at the latest stages.

The triangular matrix representation [14] is shown in figure 2.2(b) and it is based on the idea that rotations can be moved along the FFT stages. In it, rows

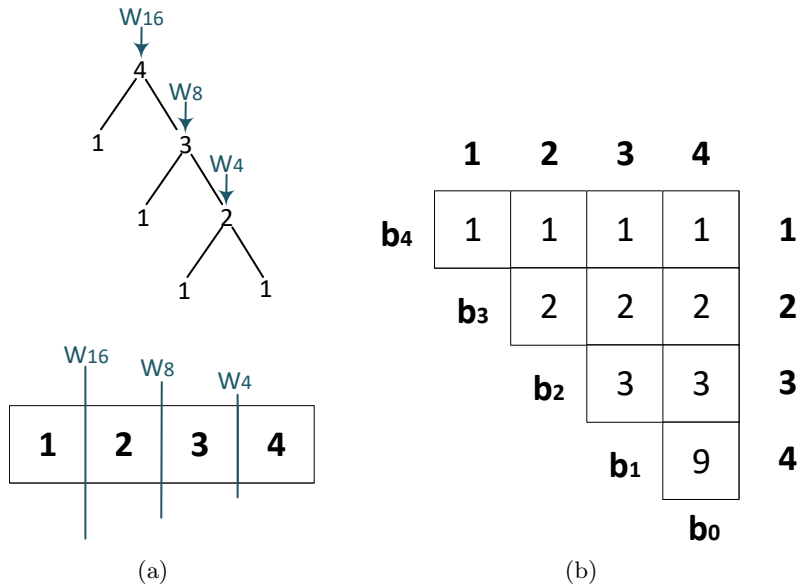


Figure 2.2: 16 point Radix-2 DIF FFT representations. (a) Binary tree and rotation distribution among stages. (b) Triangular matrix.

are numbered from left to right as $x = 1, \dots, n - 1$ and columns are numbered from top to bottom as $y = 1, \dots, n - 1$. Each element inside the matrix located in row x and column y , M_{xy} , indicates the stage where a set of rotations is placed, which is restricted to

$$x \leq M_{xy} \leq y. \quad (2.4)$$

Note that in the example of figure 2.2(b) the rotations are moved towards the earliest stages possible (DIF) which could be translated into $M_{xy} = x$. In case of a DIT distribution, the rotations will be realized the latest stages possible, corresponding in $M_{xy} = y$. In between these two possibilities, there is a variety of rotation movements allowed, which can improve performance indicators in hardware.

2.2 FFT architectures

There are numerous FFT architectures [8]. For the purpose of this Thesis, we will focus mainly on two kinds of architectures, the single-path delay feedback (SDF) [15] and the single-stream feedforward (SFF) [16].

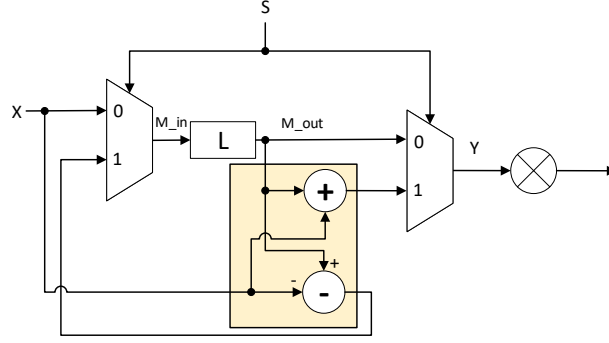


Figure 2.3: Stage of an SDF FFT architecture.

Table 2.1: SDF radix-2 values over time.

t	X	M_{in}	M_{out}	S	Y
0	X_0	X_0	—	0	—
1	X_1	X_1	—	0	—
2	X_2	X_2	—	0	—
...	...	...	...	...	...
7	X_7	X_7	—	0	—
8	X_8	$X_0 - X_8$	X_0	1	$X_0 + X_8$
9	X_9	$X_1 - X_9$	X_1	1	$X_1 + X_9$
...	...	...	...	...	...
15	X_{15}	$X_7 - X_{15}$	X_7	1	$X_7 + X_{15}$
16	X_{16}	X_{16}	$X_0 - X_8$	0	$X_0 - X_8$
17	X_{17}	X_{17}	$X_1 - X_9$	0	$X_1 - X_9$
...	...	...	...	...	...

2.2.1 SDF FFT architecture

The radix-2 SDF FFT architecture is one of the first FFT hardware architectures proposed. Each stage consists in an adder, a subtracter, two multiplexers and a buffer of length $L = 2^{n-s}$, where $n = \log_2(N)$ is the total number of stages of the FFT and s is the current stage. This architecture is shown in figure 2.3.

The value r in radix- r means the amount of inputs to the butterfly so, radix-2 means 2-input butterflies while, for example, radix-4 means 4-input butterflies. The usage of higher-radix butterflies allows for the usage of less stages and rotators, but increases the number of memories required per stage and the complexity of the rotators, so it will not be considered for this Thesis.

In table 2.1 it is shown an example of the behavior of an SDF FFT stage with $L = 8$. There, Y outputs the addition of $X_{t-8} + X_t$ while $S = 1$ and outputs the subtraction of $X_{t-8} - X_t$ while $S = 0$. The control signal S is controlled by a counter.

The main problem the SDF is that, as seen in table 2.1, the adder and subtracter of the butterfly are only used during half of the clock cycles. This means, that the

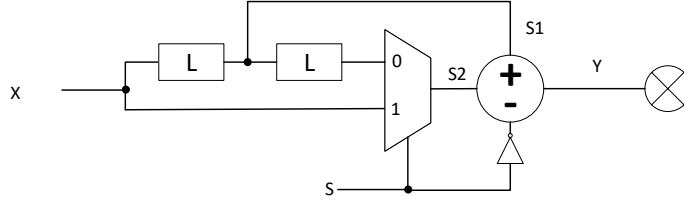


Figure 2.4: Stage of an SFF FFT architecture.

Table 2.2: SFF table of values over time.

t	X	S_1	S_2	S	Y
0	X_0	—	—	0	—
1	X_1	—	—	0	—
2	X_2	—	—	0	—
...	...	...	...	...	...
7	X_7	—	—	0	—
8	X_8	X_0	X_8	1	$X_0 + X_8$
9	X_9	X_1	X_9	1	$X_1 + X_9$
...	...	...	...	...	...
15	X_{15}	X_7	X_{15}	1	$X_7 + X_{15}$
16	X_{16}	X_0	X_8	0	$X_0 - X_8$
17	X_{17}	X_1	X_9	0	$X_1 - X_9$
...	...	...	...	...	...

usage of these components is only 50%.

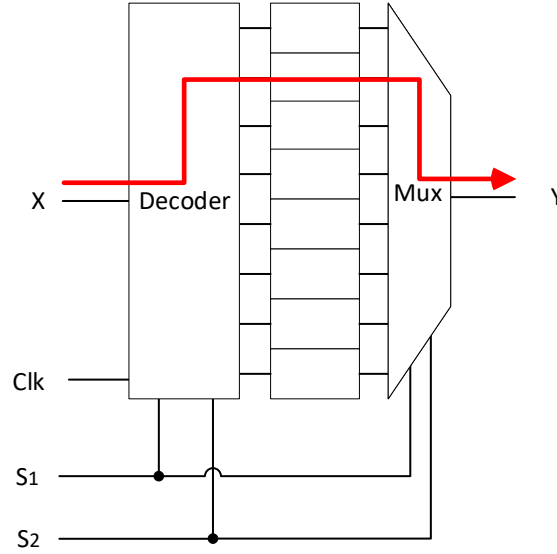
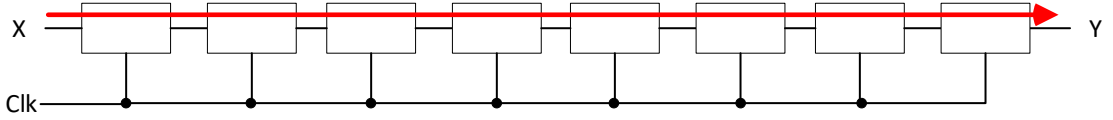
2.2.2 SFF FFT architecture

For solving the problem of the low utilization in the SDF architecture, other alternatives such as the SFF architecture [16] came to place. This architecture is shown in figure 2.4. In this architecture, only one adder/subtractor is required and it is used in all the clock cycles. The disadvantage of this architecture is that it multiplies by two the buffer size at each stage as shown in figure 2.4.

In table 2.2 we can see an example of the behavior of an SFF FFT stage with $L = 8$. There, the outputs over time are the same as in the case of the SDF case, alternating $X_{t-8} + X_t$ while $S = 1$ and $X_{t-8} - X_t$ while $S = 0$, but without requiring an adder and a subtracter.

2.2.3 Memory or shift-register based butterflies

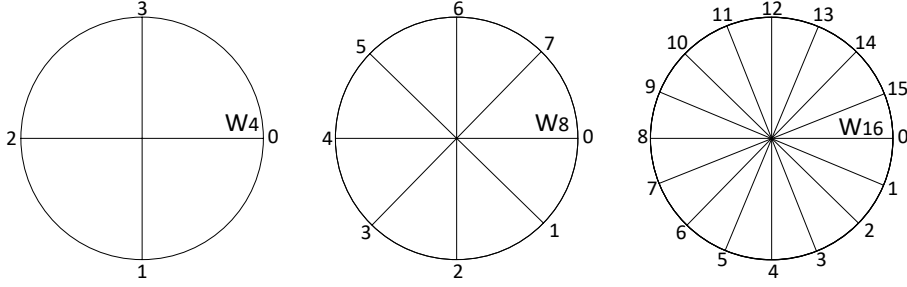
As shown, butterflies of an FFT consist in additions and subtractions of signals separated between them an interval of L clock cycles. This separation of L cycles can be achieved by storing the old inputs in shift registers or in memories, while waiting for the cycle where you have to operate the values together.

Figure 2.5: $L = 8$ memory data-flow, only impacts one address.Figure 2.6: $L = 8$ Shift Register data-flow, data moves all over the shift register.

The main difference between the usage of shift registers and memories is that, in the case of the memories, only one memory address changes every clock cycle, as shown in figure 2.5. In the case of the shift-register, all data addresses will need to change each clock cycle as shown in figure 2.6.

This behavioral difference makes recommendable the usage of shift registers for $L < 8$, as no extra control circuits for controlling the memory selection is required. Conversely, in case that $L \geq 8$, memories are recommended because the shift register power consumption grows exponentially as a function of L while in the case of a memory it is almost constant for any L , as only one address changes every clock cycle independently of the value of L [17].

About the area required, something similar happens. For $L < 16$, the shift register takes up less area, whereas for $L \geq 16$ the use of memory is more recommendable [17].


 Figure 2.7: W_4 , W_8 and W_{16} twiddle factors.

2.3 Rotators

A rotation for an angle α can be understood as a complex multiplication described as

$$\begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} C & -S \\ S & C \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}, \quad (2.5)$$

where $X + jY$ is the result of the rotation and C , S are the real and imaginary part of the rotator coefficient $C + jS$. They are calculated as

$$\begin{aligned} C &= R(\cos(\alpha) + \epsilon_c); \\ S &= R(\sin(\alpha) + \epsilon_s), \end{aligned} \quad (2.6)$$

where R is the magnitude of the coefficient and ϵ_c and ϵ_s are the quantification errors of the coefficients. The approximated total error of a rotation could be then calculated as [18]

$$\epsilon = \sqrt{\epsilon_c^2 + \epsilon_s^2}. \quad (2.7)$$

The rotation angles for the FFT architectures are determined by the twiddle factors. A W_L rotator rotates by the angles resulting from dividing the circumference in L equal parts. Examples for W_4 , W_8 and W_{16} are shown in figure 2.7. Note that rotations in a DFT architecture go in clockwise direction.

There are two main types of rotators: constant rotators and general rotators.

2.3.1 Constant rotators

Constant rotators calculate rotations by specific angles. In constant rotators, the *a priori* knowledge of the rotation angles allows for optimizing the implementations of the rotator. When the amount of angles required by a twiddle factor is small, $L \leq 32$, constant rotators are used as they have better performance thanks to

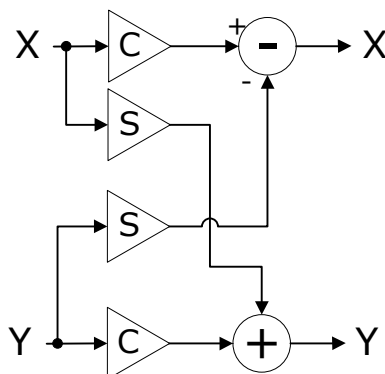


Figure 2.8: Direct multiplication strategy architecture.

their optimization. For twiddle factors which require many rotation angles $L \geq 64$, the amount of extra logic required by constant rotators makes generic rotators preferable [19].

The implementation of constant rotators can be done by using two approaches: multiplier-based implementation or coefficient selection and shift-and-add implementation.

Multiplier-based rotation

The most straightforward strategy to implement a rotator consists of a complex multiplier and a ROM memory that stores all possible rotation coefficients [20, 21]. Then, those values are multiplied by the inputs and the x and y results are crossed as shown in figure 2.8. The coefficients will be selected from the rotation memory according to a control signal that may be more or less complex depending on the re-usability of memory positions.

Coefficient selection and Shift-and-add implementation rotation

The design of constant rotators can be done as well based on two fundamental elements: the coefficient selection and the shift-and-add implementation.

The shift-and-add implementation is based on shifting right all bits of a number and adding these shifted numbers. Shifting right all bits is a common strategy for power of 2 multiplications as every bit added at the right of a binary number (0 in case of a positive value or 1 in case of a negative one) represents a multiplication by 2. This shifting does not represent any hardware cost as it only requires a relocation of bits.

Multiplications by non-power of 2 constants can be seen as a series of additions

of power of 2 multiplications. For example, a multiplication by 6 could be seen as a multiplication by 4 added to a multiplication by 2 as shown

$$\begin{aligned} x_i &= \text{"00000111"} = 7; \\ x_{out} &= 6 * x_i = 2 * x_i + 4 * x_i \\ x_{out} &= \text{"00001110"} + \text{"00011100"} = 14 + 28 = 42. \end{aligned}$$

This shift-and-add multiplication can be implemented in different ways. For example, a multiplication by 15 could be implemented as $8 + 4 + 2 + 1 = 15$ or $16 - 1 = 15$. These different ways of implementing shift-and-add rotators are based on different implementation algorithms. Examples of implementation algorithms are the HCUB [22], DiffAG [23], MAG2 [24] or the RPAG-CMM [25].

The success of previous Coefficient selection and Shift-and-add implementation rotators is mainly due to an efficient shift-and-add implementation. However, the coefficient selection has hardly been taken into account. In multiplier-based approaches the coefficients are traditionally obtained by rounding the sine and cosine components of the angle, as seen in equation 2.7. However, it has been proved that an addition-aware quantization [26] can provide better coefficient results.

The novel Combined coefficient Selection and Shift-and-add Implementation (CCSSI) [19] rotators design uses a new perspective which sets the coefficient selection and the shift-and-add implementation as equally important elements. This paradigm will be the one followed for the shift-and-add constant rotators implemented in chapter 3.

2.3.2 General rotators

General rotators can carry out rotations by any angle, which is provided as an input to the rotator. They are usually implemented by a complex multiplier or by the coordinate rotation digital computer (CORDIC) algorithm [27]. The CORDIC algorithm is a well known algorithm for calculating rotations in digital systems. Its main principle consists in breaking the rotation angle into an addition of small micro-rotations.

The rotation coefficients used in the CORDIC are $P = C + jS = 2^k + j\delta_k$, where $\delta_k \in \{-1, 1\}$ and $k = 0, \dots, M$, being M the micro-rotation stage. The corresponding angles are $\alpha_k = \tan^{-1}(S/C) = \delta_k \tan^{-1}(2^{-k})$. This is shown in figure 2.9(a).

As a result, the CORDIC algorithm breaks down the rotation angle θ into small micro-rotations like

$$\theta = \sum_{k=0}^M \alpha_k + \epsilon_\phi, \quad (2.8)$$

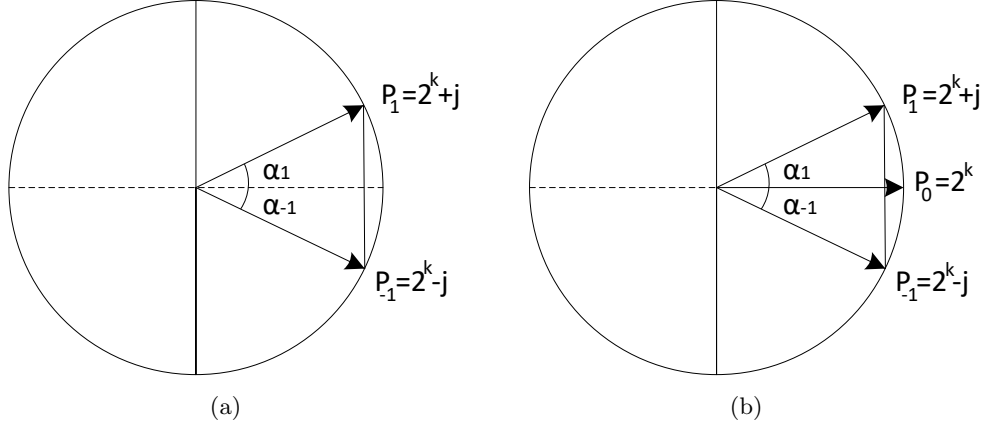


Figure 2.9: CORDIC algorithm examples. (a) Traditional CORDIC algorithm. (b) Redundant CORDIC algorithm.

where ϵ_ϕ represents the phase error. Each micro-rotation is calculated as

$$\begin{bmatrix} X_D \\ Y_D \end{bmatrix} = \begin{bmatrix} 2^k & -\delta_k \\ \delta_k & 2^k \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}, \quad (2.9)$$

where δ_k determines if the rotation is applied in a positive or negative direction and the scaling factor of the stage is $R(k) = \sqrt{2^{2k} + 1}$.

Alternative to this, there exists the redundant CORDIC [18], which allows as well the rotation by the angle $0^\circ = P_0 = 2^k$. This δ_k can acquire three possible values $\{1, 0, -1\}$. This is shown in figure 2.9(b).

Since 1965, when the CORDIC algorithm was presented for the first time, many improved versions of it have been presented such as the CORDIC II [28] or the uniformly distributed CORDIC (UD CORDIC) [29] among many others [30, 31]. For this Thesis we will focus on the UD CORDIC which is based on the idea of having the same distance between the angles. This can be achieved by modifying the angle set and allowing the elimination of the computation of the remainder angle at each stage.

The UD CORDIC used for this Thesis is implemented outside of the work done for this Thesis.

Chapter 3

Hardware implementation

This chapter explains the proposed designs for a low power consumption FFT architecture and its implementation for a 1024-point-FFT. This proposed 1024-point FFT will be compared with previous architectures explained in chapter 2 and found in the literature. For this comparison, also other architectures explained in chapter 2 will be implemented.

The FFT architectures implemented for this comparison are:

- Proposed alternative SFF architecture with memories, explained in subsection 3.1.2.
- SDF architecture with shift-registers.
- SDF architecture with memories.
- SFF architecture with shift-registers.
- SFF architecture with memories.

About the rotators, multiplier-based rotators and CCSSI rotators [19] will be implemented for the W_L twiddle factors with $L \leq 32$. For the twiddle factors with $L > 32$, the new UD CORDIC [29] will be used.

3.1 Proposed designs

As is explained in chapter 1, for achieving the 6G KPIs, new hardware architectures will be required. In this section, some new techniques will be presented for FFT architectures and rotators.

Table 3.1: $L = 8$ delay implemented with 2 $L = 4$ RAM memories behavior.

t	x	Memory 1			Memory 2			y
		$Addr_1$	WE_1	M_1	$Addr_2$	WE_2	M_2	
0	x_0	00	1	—	11	0	—	—
1	x_1	00	0	—	00	1	—	—
2	x_2	01	1	—	00	0	—	—
3	x_3	01	0	—	01	1	—	—
4	x_4	10	1	—	01	0	—	—
5	x_5	10	0	—	10	1	—	—
6	x_6	11	1	—	10	0	—	—
7	x_7	11	0	—	11	1	—	—
8	x_8	00	1	x_0	11	0	—	x_0
9	x_9	00	0	—	00	1	x_1	x_1
10	x_{10}	01	1	x_2	00	0	—	x_2
11	x_{11}	01	0	—	01	1	x_3	x_3
12	x_{12}	10	1	x_4	01	0	—	x_4
13	x_{13}	10	0	—	10	1	x_5	x_5
14	x_{14}	11	1	x_6	10	0	—	x_6
15	x_{15}	11	0	—	11	1	x_7	x_7

is performed is shown in table 3.1. In it, it is shown how memory 1 stores the even numbers while Memory 2 stores the uneven ones. Note that writing in these memories is done synchronously, which means that the input x_0 got stored in the instant $t = 1$ just before the input changes to x_1 , behaving in a similar way as a shift register.

The RAM memories used for this architecture are synchronous read and write memories provided by Europractice [9]. The behavior of these RAM memories can be seen in table 3.2, where:

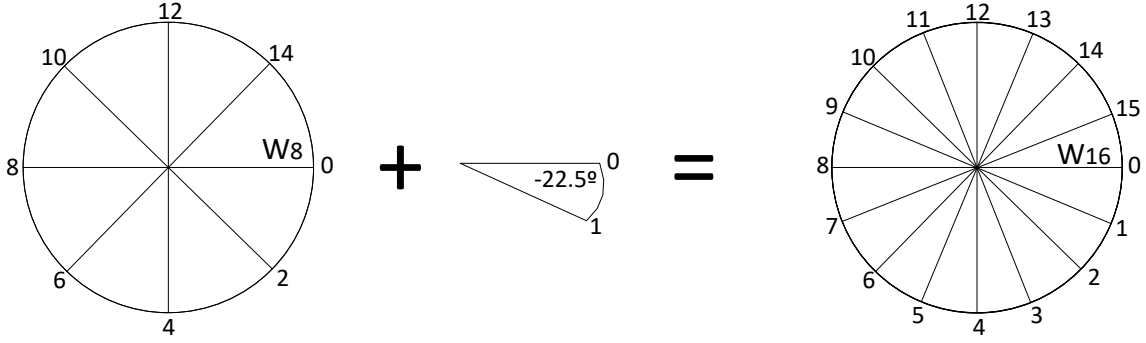
- CLK = Clock input.
- CE = Chip enable.
- WE = Write enable.
- A = Address input.
- Q = Data output.
- - = Don't care, no matter with this pin.

3.1.2 Alternative SFF FFT architecture

As shown in section 2.2.3, the usage of memories instead of shift registers is recommendable for the stages that require a buffer length of $L \geq 8$. This usage of

Table 3.3: SFF alternative, table of values over time for $L = 8$.

t	X	WE_2	WE	$Addr$	M	A	B	S_1	S_2	Y
0	X_0	1	0	0	—	0	—	—	—	—
1	X_1	1	0	1	—	0	—	—	—	—
2	X_2	1	0	2	—	0	—	—	—	—
...	...	...	...	...	...	...	...	...	...	...
7	X_7	1	0	7	—	0	—	—	—	—
8	X_8	0	1	0	—	X_8	0	X_0	X_8	$X_0 + X_8$
9	X_9	0	1	1	—	X_9	0	X_1	X_9	$X_1 + X_9$
...	...	...	...	...	...	...	...	...	...	...
15	X_{15}	0	1	7	—	X_{15}	0	X_7	X_{15}	$X_7 + X_{15}$
16	X_{16}	1	0	0	X_8	0	$\overline{X}_{16}$	X_0	$\overline{X}_{16}$	$X_0 - X_{16}$
17	X_{17}	1	0	1	X_9	0	$\overline{X}_{16}$	X_1	$\overline{X}_{17}$	$X_1 - X_{17}$
...	...	...	...	...	...	...	...	...	...	...

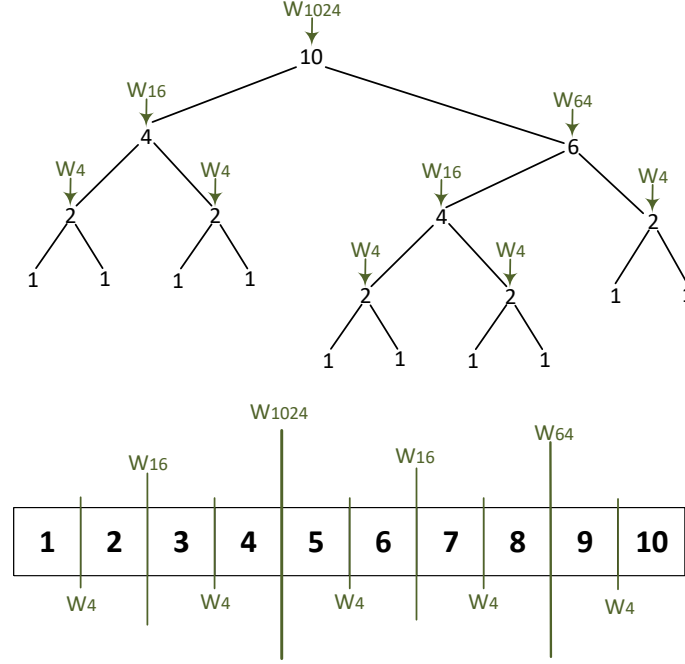

 Figure 3.4: Decimation of a rotator. Conversion of a W_{16} into a W_8 + a 22.5° angle rotation.

3.2 Rotators distribution among the FFT, Radix-2 study

According to the Cooley-Tukey algorithm [8], the DFT can be divided into smaller DFTs by exploiting the periodicity of the twiddle factor, giving as a result a DFT made up of n smaller DFTs. The rotations in between the smaller DFTs can be distributed in different ways through the FFT stages as explained in chapter 2. For the purpose of this Thesis, we have studied the Radix- 2^4 and Radix- 2^5 approaches, which correspond to the binary trees in figures 3.5 and 3.6, respectively.

From both approaches, the distribution of rotators in figure 3.7 can be extracted. As shown in the figure, the two Radix studied use different twiddle factors.

On the one side, Radix- 2^4 distribution includes the twiddle factors: W_4 ; W_{16} ; W_4 ; W_{1024} ; W_4 ; W_{16} ; W_4 ; W_{64} ; W_4 . Here, W_4 is a trivial rotator, W_{16} can be done following shift-and-add strategies or multiplier-based ones, and W_{64} and W_{1024} can be implemented with a CORDIC generic rotator as it already has too many possible rotation angles.


 Figure 3.5: Radix- 2^4 approach.

On the other side, the Radix- 2^5 approach requires W_8 and W_{32} twiddle factors instead of W_{16} and W_{64} ones. These W_8 and W_{32} twiddle factors can be implemented with shift-and-add or multiplier-based strategies. As a result, these Radix- 2^4 and Radix- 2^5 approaches end up with the rotators distribution shown in table 3.4

By comparing these two approaches from table 3.4, the amount of each type of rotator required by each Radix approach is shown in table 3.5. In it, it can be noticed that Radix- 2^4 has a higher number of trivial rotators and that its W_{16} shift-and-add rotators are simpler than the W_{32} ones required by the Radix- 2^5 . However, the usage of two CORDIC algorithms for Radix- 2^4 together with the symmetry of the rotators in the Radix- 2^5 approach (note how the twiddle factors from 1 to 5 and from 6 to 10 are the same), made Radix- 2^5 the selected approach for the FFTs developed in this Thesis.

3.3 FFT control

The FFT is controlled by a counter, which is shown in figure 3.8. Through the Radix- 2^4 FFT, there exists a big amount of delays in the signals. They are caused by the memories, shift registers and pipelines stages distributed among the FFT. This latency is shown in table 3.6. The control counter starts when the first sample arrives to the first FFT stage, and the control signals generated by the counter propagate through all the FFT taking under consideration this latency. For a good coordination with the rest of the system, it is critical to take this latency under

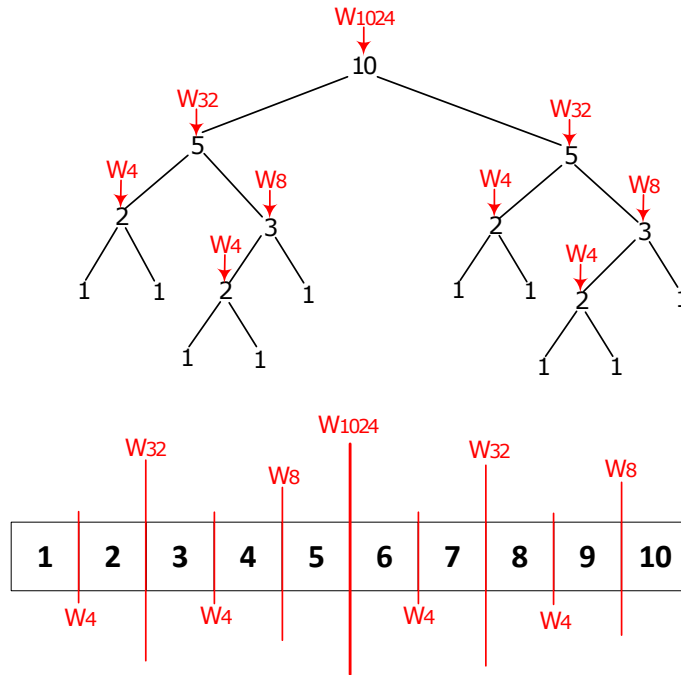
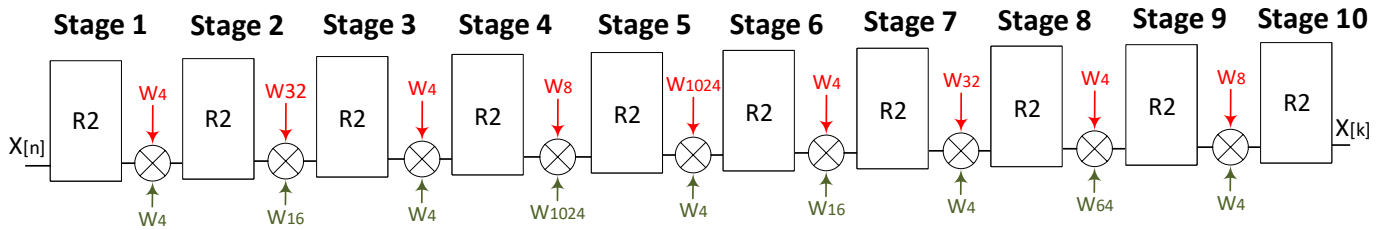

 Figure 3.6: Radix- 2^5 approach.


Figure 3.7: Distribution of the rotators among the FFT.

consideration so all components behave as expected.

3.4 Rotators control

At a first look, it could be thought that the number of possible angles to be rotated are the same as the twiddle factor sub-index (4 for a W_4 , 16 for a W_{16} ...) but actually, it can be reduced by using the decimation transformation proposed in subsection 3.1.3.

For the reduction of the number of required rotations, the triangular matrix representation already explained in chapter 2 can be used. As a result, the triangular matrix of our Radix- 2^5 FFT can be shown in figure 3.9.

The numbers inside the triangular matrix represent a set of rotations which can

Table 3.4: Radix-2⁴ and Radix-2⁵ rotators required.

Stage	Radix-2 ⁴		Radix-2 ⁵	
	W_i	Rot type	W_i	Rot type
1-2	T_4	Trivial	T_4	Trivial
2-3	W_{16}	Sh & Add	W_{32}	Sh & Add
3-4	T_4	Trivial	T_4	Trivial
4-5	C_{1024}	CORDIC	W_8	Sh & Add
5-6	T_4	Trivial	C_{1024}	CORDIC
6-7	W_{16}	Sh & Add	T_4	Trivial
7-8	T_4	Trivial	W_{32}	Sh & Add
8-9	C_{64}	CORDIC	T_4	Trivial
9-10	T_4	Trivial	W_8	Sh & Add

Table 3.5: Comparison of rotators types in Radix-2⁴ and Radix-2⁵ approaches.

	Trivial	Shift-and-add	CORDIC
Radix-2 ⁴	5	2	2
Radix-2 ⁵	4	4	1

be moved between different stages. Numbers on the right side represent the lowest stage in which the set of rotations of the corresponding line can be placed and the numbers at the top of the matrix represent the highest stage in which the set of rotators of the corresponding column can be placed. The value of each element of the triangular matrix represents then the stage in which the corresponding rotator was placed. For example, the rotator in stage 2 corresponds to the W_{32} twiddle factor and it can be placed between stages number 1 and number 4.

From this triangular matrix, the control bits for each rotator can be extracted. The way for doing so is by multiplying the bit lines in which the rotator number is present by the columns bits in which it is present.

For example, in the case of the C_{1024} CORDIC rotation (number 5), the values

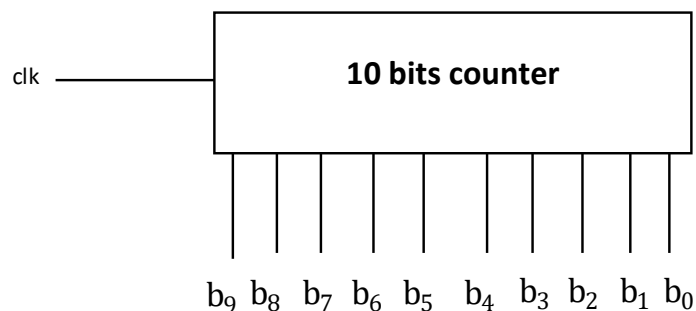


Figure 3.8: 10-bits counter in charge of the control of the FFT components.

Table 3.6: Radix-2⁵ latency between components.

Stage	Component	latency
1	Butterfly	0
	T_4	512
2	Butterfly	513
	W_{32}	770
3	Butterfly	773
	T_4	901
4	Butterfly	902
	W_8	967
5	Butterfly	969
	C_{1024}	1002
6	Butterfly	1010
	T_4	1026
7	Butterfly	1027
	W_{32}	1036
8	Butterfly	1039
	T_4	1043
9	Butterfly	1044
	W_8	1047
10	Butterfly	1049
	output	1050

to be multiplied can be extracted as shown in the figure 3.9, following equation

$$\phi_i(I) = \overbrace{\left[\begin{matrix} b_i, b_{i+1} \dots \end{matrix} \right]}^{\substack{\text{i size, complete with 0s} \\ \text{bits with i in line,}}} \cdot \overbrace{\left[\begin{matrix} b_{n-i}, b_{n-i-1} \dots \end{matrix} \right]}^{\substack{\text{i size, complete with 0s} \\ \text{bits with i in column,}}} \quad (3.1)$$

From this equation, we can get the control bits for each rotators. From the FFT of this Thesis, the table 3.7 is obtained in which it can be observed that the amount of required angles can be reduced as not all rotation angles are required. (Note that rotations go in negative direction, spinning clockwise).

As shown in table 3.7, the amount of possible angles was drastically reduced for most cases. The T_4 passes from 4 to 2, the W_8 from 8 to 4, the W_{32} from 32 to 19 and the C_{1024} from 1024 to 310.

This list allows to reduce the size of the memories required for direct multiplication-based strategies or the complexity of the shift-and-add ones, but it may increase the complexity of the value selection of the memory position as it will not be linear in most cases.

	1	2	3	4	5	6	7	8	9	
b_9	1	2	2	2	5	5	5	5	5	1
b_8		2	2	2	5	5	5	5	5	2
b_7			3	4	5	5	5	5	5	3
b_6				4	5	5	5	5	5	4
b_5					5	5	5	5	5	5
b_4						6	7	7	7	6
b_3							7	7	7	7
b_2								8	9	8
b_1									9	9
b_0										

 Figure 3.9: 1024-inputs Radix-2⁵ FFT triangular matrix.

3.5 Trivial rotators implementation

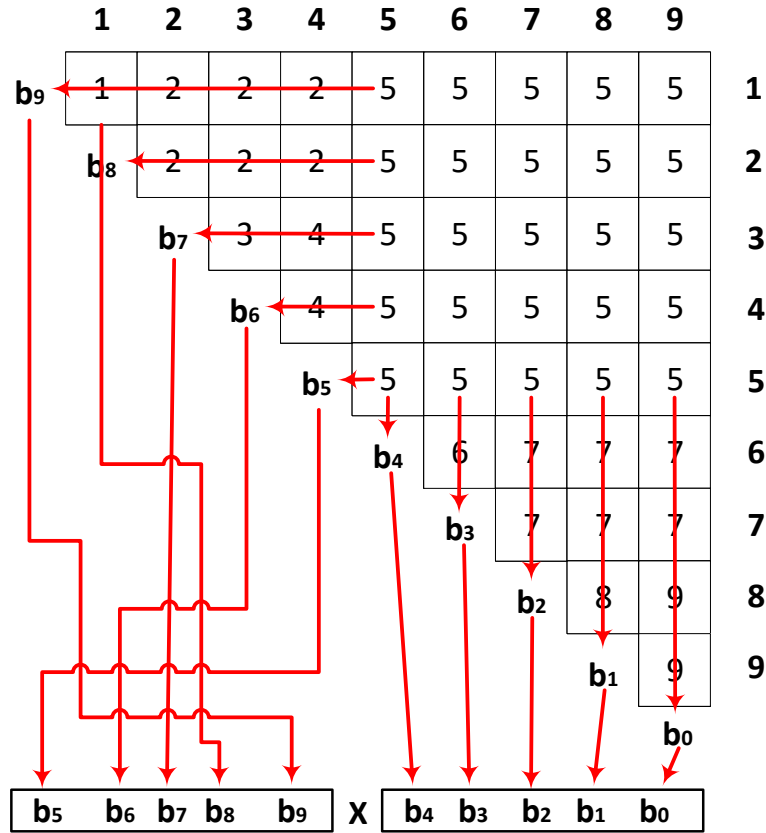
The trivial rotator T_4 is called trivial because it does not require any kind of multiplication or addition, it just requires to cross the x and y values and a sign change for calculating the rotations. For the case of our particular FFT, as extracted from the table 3.7, it will just require to rotate 0° and -90° .

The control for this rotator can be seen in table 3.8, behavior which matches a normal AND gate with b_9 & b_8 as inputs.

The architecture for this trivial rotation is shown in figure 3.11 and consisting in just two multiplexers and one buffer which changes the sign of x . This rotator is used for the W_4 twiddle factors in $\phi_1(I)$, $\phi_3(I)$, $\phi_6(I)$ and $\phi_8(I)$ as shown in table 3.7. Also, it will be part of the W_8 rotators in $\phi_2(I)$ and $\phi_7(I)$.

3.6 Multiplier-based implementations

For the purpose of this Thesis, two strategies for implementing the rotators will be used for implementing the rotators of the proposed FFT and compared. These two strategies are the Multiplier-based implementations and the CCSSI-based


 Figure 3.10: Twiddle factor n^{95} control bits from the triangular matrix.

implementation.

The multiplier-based rotator strategy was already explained in chapter 2, and it just requires four multipliers, two adders and a ROM memory. Extracted from table 3.7, it can be seen that the only rotators to be implemented with this strategy are the $\phi_4(I) / \phi_9(I)$ (corresponding to the twiddle factor W_8) and the rotators $\phi_2(I) / \phi_7(I)$ (which corresponds to the twiddle factor W_{32}).

3.6.1 The W_8 rotator implemented using multipliers

For developing a rotator for the twiddle factor W_8 , it is necessary to analyze all possible values of the control bits and check which angles and which values of C and S the rotator requires. This analysis is shown in table 3.9.

From table 3.9 we can extract the control logic for the ROM that stores all C and S possible values. In this case, it consists in a ROM of four slots of 11 bits controlled by just two AND gates, as shown in 3.12, 10 bits for the number and 1 bit for the sign (signed numbers).

Table 3.7: Rotation angles according to triangular matrix.

<i>Rotation</i>	<i>Type</i>	<i>Term₁</i>	<i>Term₂</i>	<i>n_{angles}^o</i>	<i>angles</i>
$\phi_1(I)$	T_4	b_9	b_8 0000 0000	2	0; -90
$\phi_2(I)$	W_{32}	$b_8 b_9$	$b_7 b_6 b_5$ 0 0000	18	0; -11.25; -22.5; -33.75; -45; -56.25; -67.5; -78.75; -90; -101.25; -112.5; -135; -157.5; -168.75; -202.5; -223.25
$\phi_3(I)$	T_4	b_7 00	b_6 00 0000	2	0; -90
$\phi_4(I)$	W_8	$b_6 b_7$ 00	b_5 0 000	4	0; -45; -90; -135
$\phi_5(I)$	C_{1024}	b_5 $b_6 b_7 b_8 b_9$	b_4 $b_3 b_2 b_1 b_0$	310	0; -0.35; -0.7; ... - 337.85
$\phi_6(I)$	T_4	b_4 0 0000	b_3 000	2	0; -90
$\phi_7(I)$	W_{32}	$b_3 b_4$ 0 0000	$b_2 b_1 b_0$	18	0; -11.25; -22.5; -33.75; -45; -56.25; -67.5; -78.75; -90; -101.25; -112.5; -135; -157.5; -168.75; -202.5; -223.25
$\phi_8(I)$	T_4	b_2 000 0000	b_1 0	2	0; -90
$\phi_9(I)$	W_8	$b_1 b_2$ 000 0000	b_0	4	0; -45; -90; -135

 Table 3.8: T_4 , $\phi_1(I)$ control bits.

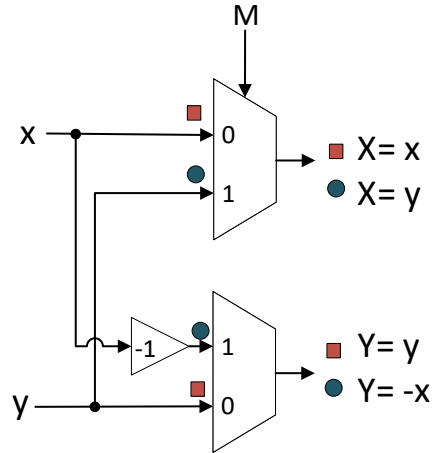
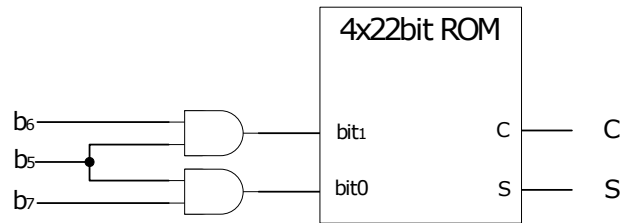
b_9	b_8	angle
0	0	0 ^o
0	1	0 ^o
1	0	0 ^o
1	1	-90 ^o

About the calculation of C and S , the following equation is used taking under consideration that $BitWidth = 11$.

$$\begin{aligned} C &= (2^{BitWidth} - 1) \cdot \cos(\phi) \\ S &= (2^{BitWidth} - 1) \cdot \sin(\phi). \end{aligned} \quad (3.2)$$

 Table 3.9: rotation $\phi_4(I)$ corresponding with the twiddle factor W_8 , control bits and angles explosion.

b_5	b_6	b_7	angle	C	S
0	0	0	0 ^o	1023	0
0	0	1	0 ^o	1023	0
0	1	0	0 ^o	1023	0
0	1	1	0 ^o	1023	0
1	0	0	0 ^o	1023	0
1	0	1	-45 ^o	723	-723
1	1	0	-90 ^o	0	-1023
1	1	1	-135 ^o	-723	-723


 Figure 3.11: T_4 Rotator for 0° and -90° rotations.

 Figure 3.12: W_8 ROM and ROM control architecture.

The approximate total error can be calculated using equation (2.7) as

$$\begin{aligned}\epsilon_c &= \frac{360}{2^{BitWidth}} = 0.175^\circ \\ \epsilon_s &= \frac{360}{2^{BitWidth}} = 0.175^\circ\end{aligned}\tag{3.3}$$

$$\epsilon = \sqrt{\epsilon_c^2 + \epsilon_s^2} = 0.248^\circ.\tag{3.4}$$

Due to the small size of the memory, it was decided to implement it using logic blocks instead of memories.

The case of $\phi_9(I)$ is similar to $\phi_4(I)$ with the only difference in the control bits that are used, as shown in table 3.10.

3.6.2 The W_{32} rotator implemented using multipliers

In the case of the W_{32} , the strategy to follow is the same as the one for the W_8 . The objective is to store all possible values of C and S and then use them to control

Table 3.10: $\phi_4(I)$, $\phi_9(I)$ equivalences of control inputs.

$\phi_4(I)$	$\phi_9(I)$
b_5	b_0
b_6	b_1
b_7	b_2

the multipliers.

All the required values of C and S are shown in table 3.11. On it, 18 possible values could be selected, being the possible values higher than $2^4=16$. As memories can only have power of 2 sizes, a memory of 32 positions will be required. About the word size, the same as W_8 will be used, 22 for storing 11 bits of C and 11 of S , which leads to the same total error $\epsilon = 0.248^\circ$.

All the pictures on this subsection make reference to the rotation $\phi_2(I)$. For referring to $\phi_7(I)$, the equivalences shown in table 3.12 can be used.

3.7 CCSSI rotators

The CCSSI rotators [19] are, as multiplication-based ones, multiplication techniques for achieving rotations. However, instead of relying in multiplier blocks, they shift bit positions (multiplications by powers of 2) and add them together for calculating the rotation.

The usage of this specific hardware reduces the area and power consumption significantly as shifts do not cost any resource and adders are smaller than multipliers. In our case, the twiddle factors W_8 and W_{32} are implemented with this strategy, which applies to the rotations $\phi_2(I)$, $\phi_4(I)$, $\phi_7(I)$ and $\phi_9(I)$ of our FFT, as shown in table 3.7.

3.7.1 The W_8 rotator implemented using shift-and-add

The CCSSI W_8 rotator works similar as the direct multiplication one, already explained in subsection 3.6.1 so it will require to rotate the angles 0° , -45° , -90° and -135° as shown in the table 3.13. For achieving these rotations, the first step is to apply the decimation transformation proposed in subsection 3.1.3, dividing the W_8 twiddle factor into a T_4 trivial rotator and a -45° rotation, as shown in figure 3.13.

Regarding the T_4 rotator, it will be implemented exactly the same way as the one located in section 3.6 as both rotate the same angles.

The complete rotator can be seen in figure 3.14. In it, the T_4 rotator is shown

Table 3.11: W_{32} , $\phi_2(I)$ control bits and angles explosion.

n°	$Term_1$		$Term_2$			angle	C	S
	b_9	b_8	b_7	b_6	b_5			
1°	0	0	0	0	0	0°	1023	0
2°	0	0	0	0	1	0°	1023	0
3°	0	0	0	1	0	0°	1023	0
4°	0	0	0	1	1	0°	1023	0
5°	0	0	1	0	0	0°	1023	0
6°	0	0	1	0	1	0°	1023	0
7°	0	0	1	1	0	0°	1023	0
8°	0	0	1	1	1	0°	1023	0
9°	0	1	0	0	0	0°	1023	0
10°	0	1	0	0	1	-22.5°	945	-391
11°	0	1	0	1	0	-45°	723	-723
12°	0	1	0	1	1	-67.5°	391	-945
13°	0	1	1	0	0	-90°	0	-1023
14°	0	1	1	0	1	-112.5°	-391	-945
15°	0	1	1	1	0	-135°	-723	-723
16°	0	1	1	1	1	-157.5°	-945	-391
17°	1	0	0	0	0	0°	1023	0
18°	1	0	0	0	1	-11.25°	1003	-200
19°	1	0	0	1	0	-22.5°	945	-391
20°	1	0	0	1	1	-33.75°	851	-568
21°	1	0	1	0	0	-45°	723	-723
22°	1	0	1	0	1	-56.25°	568	-851
23°	1	0	1	1	0	-67.5°	391	-945
24°	1	0	1	1	1	-78.75°	200	-1003
25°	1	1	0	0	0	0°	1023	0
26°	1	1	0	0	1	-33.75°	851	-568
27°	1	1	0	1	0	-67.5°	391	-945
28°	1	1	0	1	1	-101.25°	-200	-1003
29°	1	1	1	0	0	-135°	-723	-723
30°	1	1	1	0	1	-168.75°	-1003	-200
31°	1	1	1	1	0	-202.5°	-945	391
32°	1	1	1	1	1	-236.25°	-568	851

first controlled by the control bit M_1 and the -45° rotation implemented following shift-and-add strategies can be seen second, controlled by the bit M_0 . The $\ll n$ blocks in the figure represent n bits right shifted, which equals a multiplication by 2^n .

3.7.2 The W_{32} rotator implemented using shift-and-add

The twiddle factor W_{32} corresponds to the rotations $\phi_2(I)$ and $\phi_7(I)$ from table 3.7. The rotation angles for the W_{32} twiddle factor rotator are shown in table 3.14. Note that for the T_4 and W_8 rotators, all angles required were smaller than 180° . However, for the W_{32} it is exceeded in some cases, which leads to the necessity

Table 3.12: $\phi_2(I)$, $\phi_7(I)$ equivalences of control inputs.

$\phi_2(I)$	$\phi_7(I)$
b_8	b_3
b_9	b_4
b_7	b_2
b_6	b_1
b_5	b_0

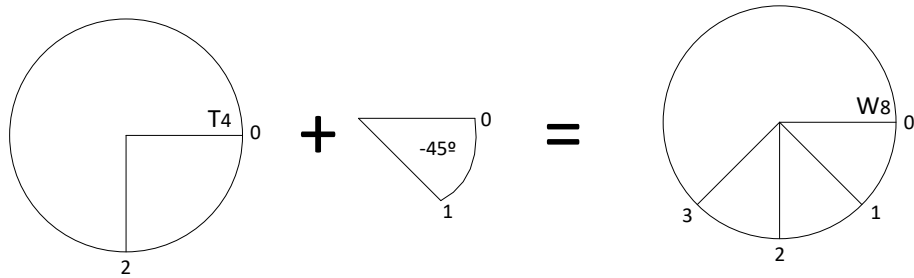
 Table 3.13: W_8 , $\phi_4(I)$ control bits.

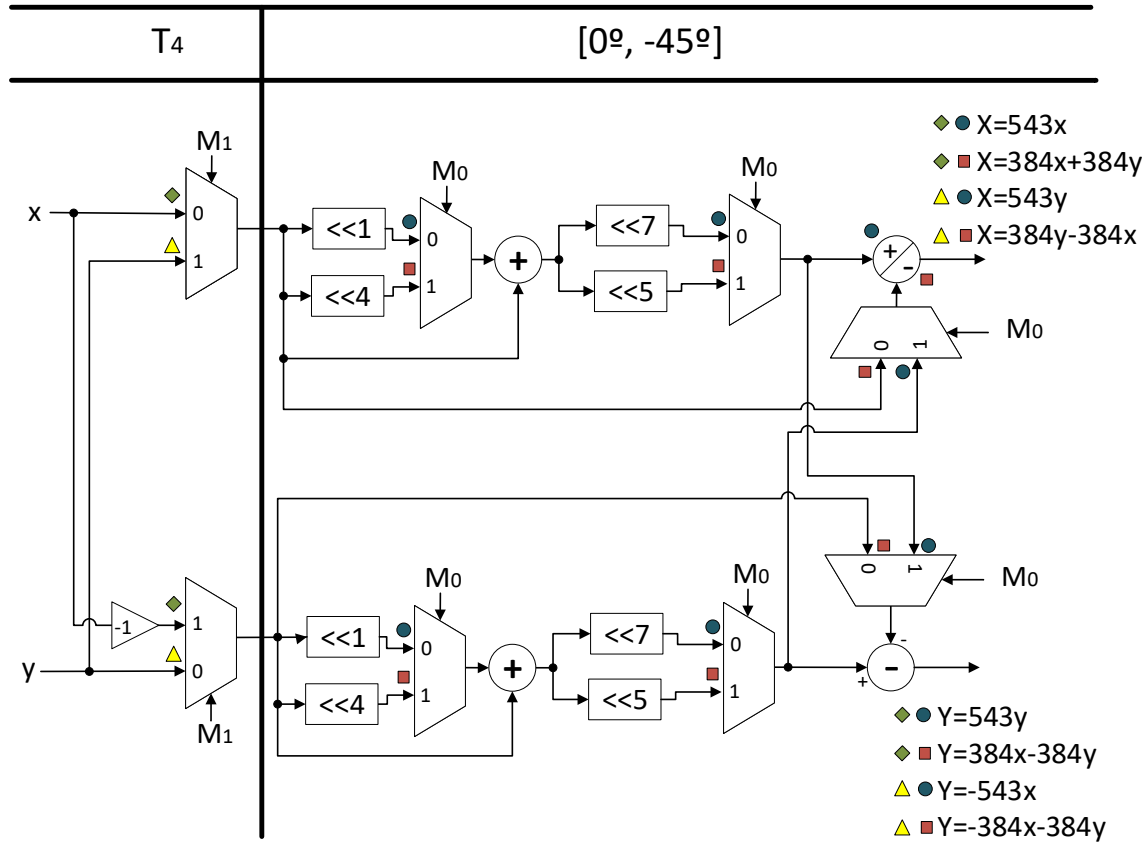
b_5	b_6	b_7	M_1	M_0	X	Y	angle
0	0	0	0	0	$543x$	$543y$	0°
0	0	1	0	0	$543x$	$543y$	0°
0	1	0	0	0	$543x$	$543y$	0°
0	1	1	0	0	$543x$	$543y$	0°
1	0	0	0	0	$543x$	$543y$	0°
1	0	1	0	1	$384x + 384y$	$384x - 384y$	-45°
1	1	0	1	0	$543y$	$-543x$	-90°
1	1	1	1	1	$-384x + 384y$	$-384x - 384y$	-135°

of a different T_4 rotator than the one explained in section 3.5.

For implementing it, the decimation transformation proposed in subsection 3.1.3 is used, splitting the W_{32} twiddle factor into a W_8 and a 0° / -11.25° / -22.5° / -33.75° rotation. This split is shown in figure 3.15

After decimating the twiddle factor, it can be optimized by adding to the 0° / -11.25° / -22.5° / -33.75° rotation a constant, moving the angles to rotate symmetrically to the X axis as shown in the figure 3.16. The advantage of this implementation is that the 4-angle rotation changes into a 2-angle one together with one mirror hardware by exploiting the symmetry around the X axis. This mirror can be implemented by just exchanging the x input for the y input and the x output for the y output, changing the rotation from positive to negative angles. About the constant we are generating by doing this, it can be easily compensated by adding a negative constant to $\phi_2(I)$ and a positive one to $\phi_7(I)$ as shown in the figure 3.16.


 Figure 3.13: W_8 decimation into a T_4 and a -45° rotation.


 Figure 3.14: W_8 shift-and-add architecture.

As a result of the transformations made in the figure 3.16, the resulting W_{32} rotator is made up of an W_8 rotator together with a $1/64$ or $3/64$ (equivalent to 5.635° and 16.875° respectively) rotator with a mirror allowing to rotate positive or negative angles.

W_8 , first part of the W_{32}

The W_8 rotator is similar to the one explained in the section 3.7.1 but with a small modification. The one used for $\phi_4(I)$ and $\phi_9(I)$ is slightly simplified as it does not required the possibility of rotating -180° or higher. However, for the rotators $\phi_2(I)$ and $\phi_7(I)$ this possibility is required. This is shown in table 3.7.

For achieving it, an slightly more complex approach is required, which includes 3 control bits for spinning -180° , -90° and -45° , respectively. The values of the angles to be rotated depending on the control inputs are shown in table 3.15. This W_8 architecture implemented for achieving those angles is shown in figure 3.17.

Table 3.14: W_{32} , $\phi_2(I)$ control bits for shift-and-add rotator.

n°	$Term_1$		$Term_2$			angle	angle decomposition
	b_9	b_8	b_7	b_6	b_5		
1°	0	0	0	0	0	0°	0°
2°	0	0	0	0	1	0°	0°
3°	0	0	0	1	0	0°	0°
4°	0	0	0	1	1	0°	0°
5°	0	0	1	0	0	0°	0°
6°	0	0	1	0	1	0°	0°
7°	0	0	1	1	0	0°	0°
8°	0	0	1	1	1	0°	0°
9°	0	1	0	0	0	0°	0°
10°	0	1	0	0	1	-22.5°	22.5°
11°	0	1	0	1	0	-45°	45°
12°	0	1	0	1	1	-67.5°	$45^\circ + 22.5^\circ$
13°	0	1	1	0	0	-90°	90°
14°	0	1	1	0	1	-112.5°	$90^\circ + 22.5^\circ$
15°	0	1	1	1	0	-135°	$90^\circ + 45^\circ$
16°	0	1	1	1	1	-157.5°	$90^\circ + 45^\circ + 22.5^\circ$
17°	1	0	0	0	0	0°	0°
18°	1	0	0	0	1	-11.25°	11.25°
19°	1	0	0	1	0	-22.5°	22.5°
20°	1	0	0	1	1	-33.75°	$22.5^\circ + 11.25^\circ$
21°	1	0	1	0	0	-45°	45°
22°	1	0	1	0	1	-56.25°	$45^\circ + 11.25^\circ$
23°	1	0	1	1	0	-67.5°	$45^\circ + 22.5^\circ$
24°	1	0	1	1	1	-78.75°	$45^\circ + 22.5^\circ + 11.25^\circ$
25°	1	1	0	0	0	0°	0°
26°	1	1	0	0	1	-33.75°	$22.5^\circ + 11.25^\circ$
27°	1	1	0	1	0	-67.5°	$45^\circ + 22.5^\circ$
28°	1	1	0	1	1	-101.25°	$90^\circ + 11.25^\circ$
29°	1	1	1	0	0	-135°	$90^\circ + 45^\circ$
30°	1	1	1	0	1	-168.75°	$90^\circ + 45^\circ + 22.5^\circ + 11.25^\circ$
31°	1	1	1	1	0	-202.5°	$180^\circ + 22.5^\circ$
32°	1	1	1	1	1	-236.25°	$180^\circ + 45^\circ + 11.25^\circ$

5.625°/16.875° rotation and constant, second part of the W_{32}

This second part of the W_{32} rotator is made up of two main components, i.e., the rotation of 5.625° or 16.875° and a selector in charge of inverting input and output, allowing this block to rotate $\pm 5.625^\circ/\pm 16.875^\circ$. The result is shown in figure 3.18. The control inputs and resulting angles are shown in 3.16

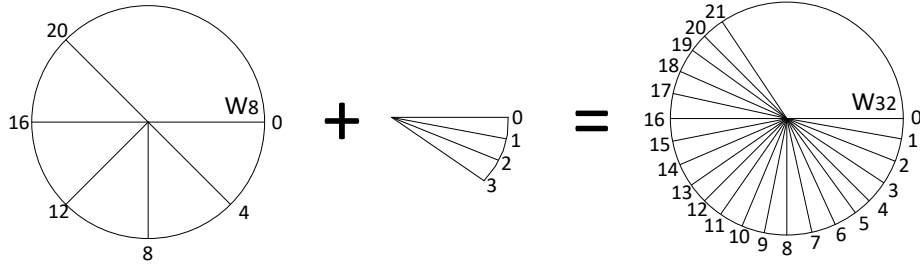
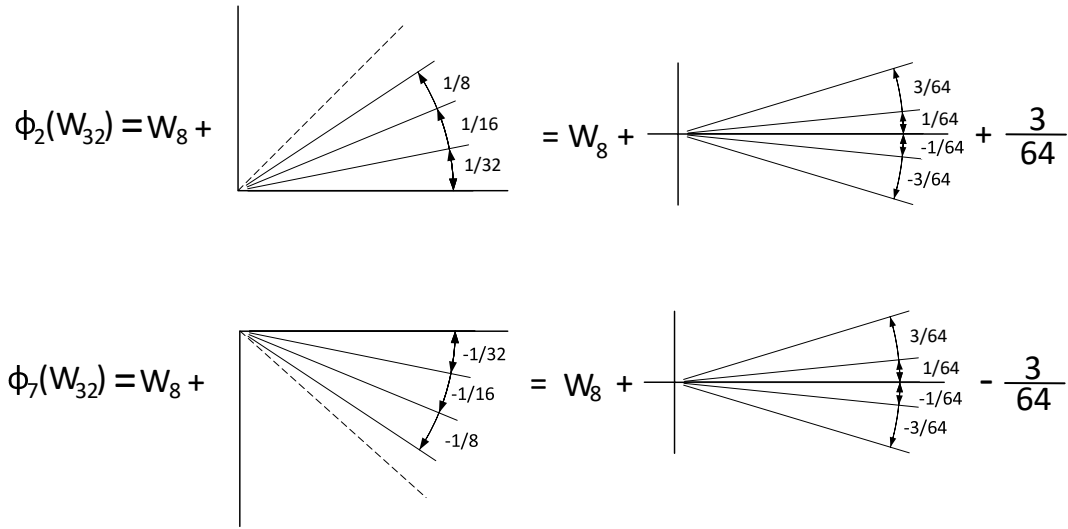

 Figure 3.15: W_{32} decimation into a W_8 + a $-33.75^\circ/-22.5^\circ/-11.25^\circ$ rotation.

 Figure 3.16: W_{32} modifications following decimation, reduction, center, move and merge transformations [1].

 Table 3.15: W_8 control bits and angles resultant.

M_4	M_3	M_2	X	Y	angle
0	0	0	$543x$	$543y$	0°
0	0	1	$384x + 384y$	$384x - 384y$	-45°
0	1	0	$543y$	$-543x$	-90°
0	1	1	$-384x + 384y$	$-384x - 384y$	-135°
1	0	0	$543y$	$543x$	-180°
1	0	1	$-384x + 384y$	$384x + 384y$	-225°
1	1	0	$-543y$	$-543x$	-270°
1	1	1	$384y + 384x$	$384y - 384x$	-315°

 Table 3.16: $\pm 5.625^\circ/\pm 16.875^\circ$ Rotator control and coefficients.

M_1	M_0	X	Y	angle
0	0	$80x - 8y$	$24y + 8x$	0°
0	1	$79x - 24y$	$79y + 24x$	-45°
1	0	$80y - 8x$	$24x + 8y$	-90°
1	1	$79y - 24x$	$79x + 24y$	-135°

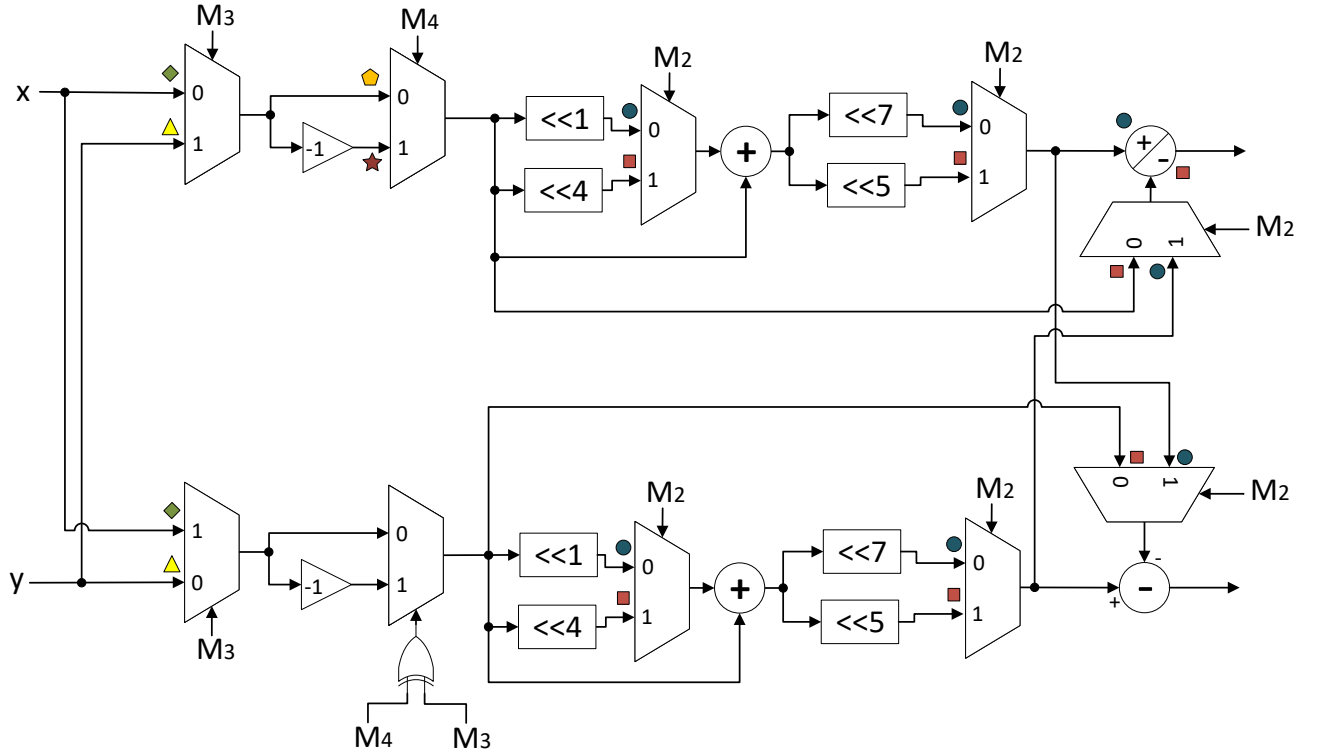


Figure 3.17: W_8 shift-and-add architecture allowing as well 180° rotations.

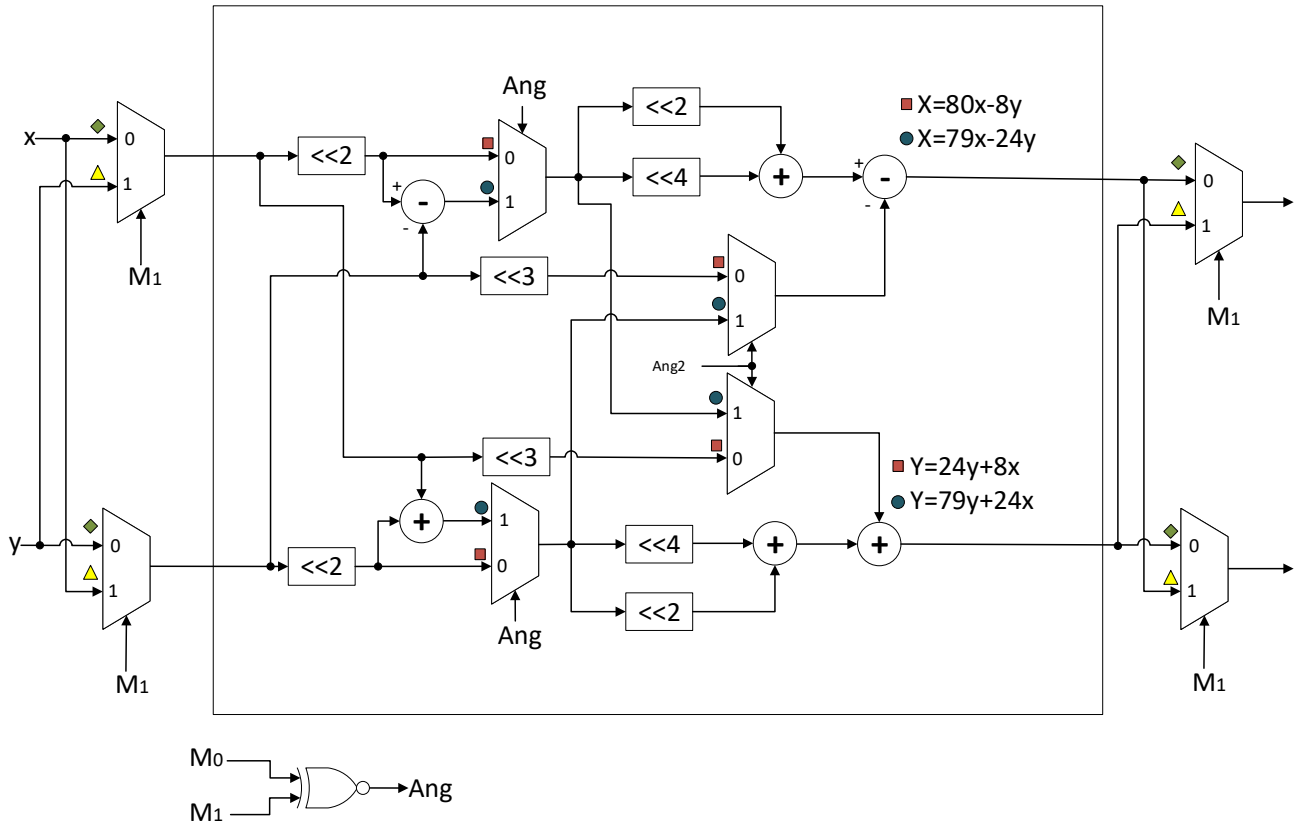


Figure 3.18: $\pm 5.625^\circ / \pm 16.875^\circ$ Rotator architecture.

Chapter 4

Experimental results and comparison

The synthesis is the step of transforming VHDL hardware description code into standard logic cells for a particular chip technology. Those standard cells are logic gates flip-flops, full adders... with a particular speed, size and power consumption. The next step after synthesis is the physical design of the chip, which involves the placement of the cells, the routing of the signals and the pin-out from the chip to the outside world. The physical design of the chip is out of the scope for this Thesis, as this Thesis aims to compare the proposed architectures and not a real design for fabrication. As a result, this Thesis will base all its comparisons in the synthesized designs.

In this chapter, all possible components for building a 1024-point-FFT (FFT stages and rotators) are analyzed in terms of area, power consumption and maximum clock frequency. For this analysis, all the blocks have been synthesized into the TSMC 40 nm technology standard logic cells using the Synopsis Design Compiler.

4.1 FFT stages comparison

As already exposed in chapter 3, the FFT stages implemented are:

1. SDF architecture using shift registers as buffers.
2. SFF architecture using shift registers as buffers.
3. SDF architecture using memories implemented with flip-flops as buffers.
4. SFF architecture using memories implemented with flip-flops as buffers.

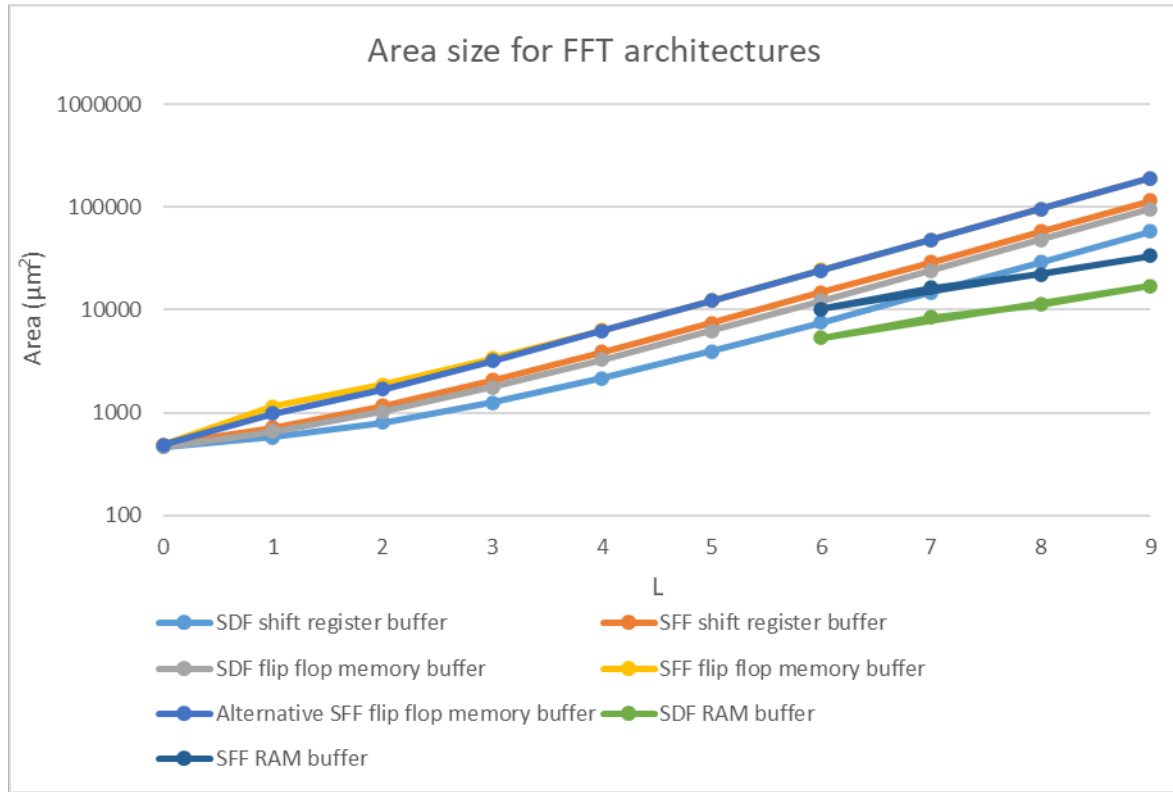


Figure 4.1: Area of different FFT stages (without rotator) as a function of the buffer length, L .

5. Alternative SFF architecture using memories implemented with flip-flops as buffers.
6. SDF architecture using RAM memories with the architecture proposed in subsection 3.1.1 as buffers.
7. SFF architecture using RAM memories with the architecture proposed in subsection 3.1.1 as buffers.

The architectures with shift registers as buffers or memories implemented with flip-flops, are synthesized directly. This means that the architectures described in section 2.2 and subsection 3.1.2 are coded in VHDL with the buffers and synthesized all together.

For the architectures based on RAM memory buffers proposed in subsection 3.1.1, the butterflies are synthesized using the Design Compiler whereas the RAM memories are provided by Europractice. As Europractice RAM memories are provided in Verilog and in fabrication formats, they cannot be analyzed inside the Design Compiler, but can be simulated together for ensuring a correct behavior. This means that the power consumption, the area and the frequency are analyzed independently

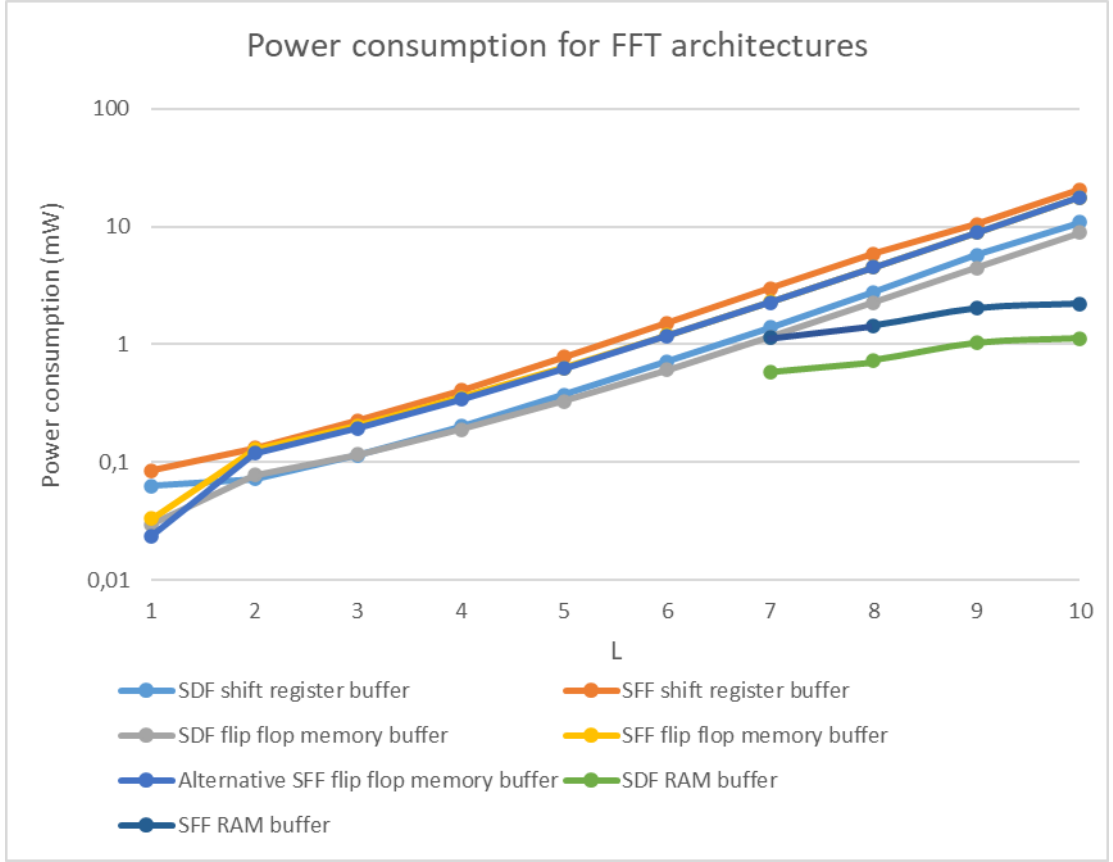


Figure 4.2: Power consumption of different FFT stages (without rotator) as a function of the buffer length, L .

for the butterfly and the buffers and then added together. The RAM memories provided by Europractice require at least 32 positions. This mean that RAM buffers can only be applied for $L \geq 6$ as they require 2×32 positions RAM for making a RAM buffer of length 64.

The synthesis can be done with different restrictions and parameters. All synthesizes for this Thesis were done pursuing the minimum power consumption constraint.

For measuring the power consumption, a frequency of 100 MHz is taken as reference, being the inputs to the each FFT stage random input values. These values are complex, with 16 bits for the real part and 16 bits for the imaginary part.

The result for all different stages can be seen in table 4.1 and table 4.2. Figure 4.1 compares the area of these FFT stages and figure 4.2 compares the power consumption.

By analyzing figures 4.1 and and 4.2, it can be observed that SDF strategies have in all cases lower power consumption and area than their equivalent SFF implementation, with an increasingly gap for high L . Note that the graphs are in logarithmic

scale, so the difference shown is being powered by the same factor.

This performance difference between SDF and SFF equivalent strategies is caused due to the contribution to the power consumption and the area of the buffers. The SDF only requires the usage of one buffer while the SFF requires two causing that SFF strategies have around double area and power consumption than the SDFs.

In terms of maximum frequency, SDF strategies are slightly slower than SFF ones. However, this could be solved by adding pipeline inside the stages.

For $L \geq 3$ the buffers using memories implemented with flip-flops have around 5% less power consumption for $L = 3$ and 18% less power consumption for $L = 9$ than those implemented with shift registers. However, the area required for this strategy is larger in all cases having around 40% more area for $L = 3$ and 65% more area for $L = 9$.

The performance difference between the alternative SFF proposed and the previous SFF implemented with buffers using memories implemented with flip-flops is small, but slightly in favor of the alternative SFF proposed.

The RAM buffer architecture proposed has better performance than other buffers for all values of L available, from $L = 6$ to $L = 9$. The performance improvement for the RAM memory architecture divides by three the power consumption for $L = 6$ compared to other strategies, and by 10 for $L = 9$. In the case of the area, the usage of buffers based on RAM memories reduce around 30% the area for $L = 6$ and 70% the area for $L = 9$ compared to shift registers.

The best power consumption buffer architecture for an SDF stage architecture depends on the value of L . For $L < 3$, the shift register buffers achieve best power consumption. For $3 \leq L \leq 5$, the best power consumption buffer are the memories implemented with flip-flops. For $L > 5$, the best power consumption is achieved by the RAM buffer architectures.

4.2 Rotators comparison

For the elaboration of the Radix-2⁵ 1024-point-FFT, W_8 and W_{32} rotators are required. The implementation of these rotators has been done following two strategies: Multiplier-based implementation and CCSSI implementation.

The CCSSI W_8 and W_{32} rotators were described in VHDL and later synthesized with Design Compiler. Similarly, the W_8 multiplier-based rotator was described in VHDL and later synthesized together with the 4-address ROM as it was too small for using a real ROM memory.

As the W_{32} multiplier-based rotator requires a memory of 32 positions, the mul-

multiplier and the ROM were analyzed separately. The multiplier was synthesized with the Design Compiler, directly while the ROM was ordered to Europractice. The area, power consumption and maximum clock frequency have been obtained as the combination of the synthesized values from the multiplier and the performance values extracted from the behavioral characteristics of the ROM.

A comparison of the performance values of the W_8 and W_{32} rotators following both strategies are shown in table 4.3. A frequency of 100 MHz is taken as reference for the power consumption measurement, being the inputs to the each rotator random complex values with 16 bits for the real part and 16 bits for the imaginary part.

As shown in table 4.3, CCSSI strategies have better performance in area, maximum clock frequency and power consumption. This performance gap is specially important in the W_8 rotator, being the power consumption of the multiplier-based rotator 6 times the power consumption of the CCSSI rotator.

For the W_{32} rotator, it is important to mention that the CCSSI rotator has 3 lines of pipeline which significantly increases the maximum frequency of the implementation but it also adds three cycles of latency that the multiplier-based strategy does not have.

As the performance of the multiplier-based rotators is significantly lower than the performance of the CCSSI ones, only the CCSSI rotators are taken under consideration for the elaboration of the complete FFTs in section 4.3.

4.3 Full FFTs comparison

The proposed Radix-2⁵ 1024-point-FFT has 10 stages, four W_4 trivial rotators, two W_8 and W_{32} CCSSI rotators and one UD CORDIC general rotator. In addition to this, it also requires a counter for the control and a series of pipeline stages for the data and control lines.

In this section, full FFTs are implemented using the different FFT architectures analyzed in section 4.3.

Apart from these FFTs, a low power consumption SDF and a low power consumption SFF are proposed based on the results of section 4.1. The proposed low power SDF uses shift register buffers for stages with $L < 3$, buffers using memories implemented with flip-flops for stages with $3 \geq L \geq 5$, and the proposed RAM buffers for the stages with $L > 5$. Regarding the proposed low power SFF, it uses shift register buffers for the first stage, the proposed alternative SFF with buffers using memories implemented with flip-flops for stages with $1 \geq L \geq 5$ and the proposed RAM buffers for the stages with $L > 5$.

The comparison of these Radix-2⁵ 1024-point-FFTs is shown in table 4.4. In it, it can be observed that the power consumption of the proposed SDF FFT is significantly lower than the power consumption in all the other implementations, being more than two times lower than the power consumption of the SDF implemented with shift registers and 5.5 times lower than in the case of the SFF with shift registers.

4.4 Comparison with previous serial FFTs on ASICs

Table 4.5 compares the proposed FFT implementation with previous serial FFTs on ASICs. In the table, area and power consumption are normalized [17] to 40 nm, 1.1V and 100MHz as

$$\text{Norm. Area} = \frac{\text{Area}}{(\text{Tech.}/40 \text{ nm})^2} \quad (4.1)$$

$$\text{Norm. Power} = \frac{\text{Power Consumption}}{(\text{Tech.}/40 \text{ nm}) \times (f_{CLK}/100 \text{ MHz}) \times (V_{DD}/1.1V)^2} \quad (4.2)$$

The proposed architecture improves the power consumption compared with previous 1024-point SDF [32], 64-point SDF [33] and 2048-point SDF [34]. At the same time, it achieves better area than previous 1024-point SDF [32] and 1024-point SC [35].

This last SC [35] architecture consists on a SDF architecture but re-ordering the butterflies among the stages. This re-order allows a reduction of the length of the internal buffers for each stage, achieving better power consumption than our proposed SDF architecture. A future step for this Thesis would be to implement an SC architecture using the RAM memory buffers proposed in subsection 3.1.1.

Table 4.1: Comparison of FFT stages for low values of L .

FFT architecture	Area (μm^2)	f_{max} (MHz)	Power consumption (mW)
$L = 0$			
SDF with registers	462	550	0.06
SFF with registers	487	641	0.09
$L = 1$			
SDF with registers	574.9	550	0.07
SFF with registers	713	641	0.13
SDF with flip-flop memories	660	476	0.08
SFF with flip-flop memories	1146	395	0.13
Alternative SFF with flip-flop memories	979	413	0.12
$L = 2$			
SDF with registers	800.7	550	0.115
SFF with registers	1164	641	0.225
SDF with flip-flop memories	1023	465	0.12
SFF with flip-flop memories	1870	390	0.21
Alternative SFF with flip-flop memories	1703	400	0.20
$L = 3$			
SDF with registers	1252	550	0.20
SFF with registers	2068	641	0.41
SDF with flip-flop memories	1765	455	0.19
SFF with flip-flop memories	3355	385	0.35
Alternative SFF with flip-flop memories	3188	386	0.34
$L = 4$			
SDF with registers	2155	550	0.373
SFF with registers	3874	641	0.78
SDF with flip-flop memories	3284	444	0.33
SFF with flip-flop memories	6393	380	0.63
Alternative SFF with flip-flop memories	6226	374	0.62
$L = 5$			
SDF with registers	3962	550	0.72
SFF with registers	7487	641	1.52
SDF with flip-flop memories	6275	435	0.61
SFF with flip-flop memories	12375	375	1.19
Alternative SFF with flip-flop memories	12208	362	1.18

Table 4.2: Comparison of FFT stages for high values of L .

FFT architecture	Area (μm^2)	f_{max} (MHz)	Power consumption (mW)
$L = 6$			
SDF with registers	7574	550	1.4
SFF with registers	14712	641	2.98
SDF with flip-flop memories	12265	425	1,16
SFF with flip-flop memories	24354	371	2,30
Alternative SFF with flip-flop memories	24187	351	2,29
SDF with RAM memories	5334	420	0.58
SFF with RAM memories	10231	455	1.14
$L = 7$			
SDF with registers	14800	550	2.77
SFF with registers	29163	641	5.87
SDF with flip-flop memories	24250	417	2.27
SFF with flip-flop memories	48324	366	4.51
Alternative SFF with flip-flop memories	48157	341	4.5
SDF with RAM memories	8457	420	0.74
SFF with RAM memories	16478	455	1.45
$L = 8$			
SDF with registers	29250	550	5.77
SFF with registers	58064	641	10.47
SDF with flip-flop memories	48184	408	4.49
SFF with flip-flop memories	96193	362	8.95
Alternative SFF with flip-flop memories	96026	332	8.94
SDF with RAM memories	11339	419	1.03
SFF with RAM memories	22242	453	2.03
$L = 9$			
SDF with registers	58152	550	10.85
SFF with registers	115867	641	20.54
SDF with flip-flop memories	95986	400	8.93
SFF with flip-flop memories	191797	357	17.83
Alternative SFF with flip-flop memories	191631	323	17.82
SDF with RAM memories	17137	418	1.12
SFF with RAM memories	33838	452	2.22

Table 4.3: Comparison of rotators.

Rotators implementation	Area (μm^2)	f_{max} (MHz)	Power consumption (mW)	Latency cycles
W_8 rotator				
Multiplier-based with registers	2437	305	1.36	0
CCSSI with registers	666	356	0.26	0
W_{32} rotator				
Multiplier-based with registers	4945	230	1.59	0
CCSSI with registers	2358	369	0.82	3

Table 4.4: Expansion of 1024-point FFT architectures.

1024-point FFT architecture	Area (μm^2)	f_{max} (MHz)	Power consumption (mW)
SDF FFT with registers	131464	356	25.20
SFF FFT with registers	246080	356	45.90
SDF FFT with flip-flop memories	206634	356	21.10
SFF FFT with flip-flop memories	386293	356	39.10
Alternative SFF FFT with flip-flop memories	397272	323	38.9
Proposed SDF FFT	67910	356	7.73
Proposed SFF FFT	120062	356	12.3

Table 4.5: Comparison of serial FFTs implemented on ASICs.

	Proposed	[35]	[32]	[33]	[34]
FFT Size	1024	1024	1024	64	2048
Radix	2^5	2	2^4	2^3	2^2
Architecture	SDF	SC	SDF	SDF	SDF
Word length	16	16	16	16	10
Technology (nm)	40	55	65	180	350
Voltage (V)	1.1	0.9	1.1	-	2.7
Area (mm ²)	0.068	0.150	0.190	0.47	7.58
Norm. Area	0.068	0.079	0.072	0.02	0.10
Power (mW)	7.73	8.0	17.1	29.7	526
	@100 MHz	@200 MHz	@50 MHz	@50 MHz	@50 MHz
Norm. Power	7.73	4.35	21.0	-	19.95

Chapter 5

Conclusion and future work

5.1 Conclusion

The objectives of this Master's Thesis have been accomplished. Multiple 1024-point-FFTs using different architectures have been implemented for a TSM 40 nm ASIC technology, pursuing the least power consumption possible. In this chapter, the different conclusions obtained from this Thesis are explained.

For all FFT stages in an ASIC design, the performance of the FFT stage is driven by the performance of the buffer delays. This impact of the buffers makes SDF architectures have half the area and power consumption of SFF ones as they require just one buffer instead of two.

In this Thesis, three types of buffers were considered: Shift register buffers, buffers with memories implemented with flip-flops and the proposed synchronous RAM buffers. These three buffers were compared for SDF and SFF architectures for different values of L .

For SDF architecture and $L < 3$, the buffer with the least power consumption is the shift register buffer. For $3 \geq L \geq 5$, the least power consumption is achieved by the memories implemented with flip-flops. For $L > 5$, the best power consumption is achieved by the RAM buffer. About the area, buffers using memories implemented with flip-flops require the biggest area. Shift register buffers have the best area size for $L < 6$ and the proposed RAM architecture achieves the best area for $L \geq 6$.

The alternative SFF architecture has less power consumption than the SFF implemented with shift registers. Also, it has slightly less power consumption than previous SFF approaches. However, it has more area and power consumption than all SDF architectures.

Apart from the different FFT architectures, two types of constant rotators strate-

gies were analyzed: Multiplier-based implementation and CCSSI implementation. These two techniques have been implemented in W_8 and W_{32} rotators, achieving CCSSI implementations significantly better performance results than multiplier-based ones.

5.2 Future work

For this Thesis, a RAM memory buffer architecture was proposed. This architecture has better performance results than any other buffer architecture analyzed. As there are many kinds of RAM memories, future research for different RAM types (asynchronous read and write, single port...) based on the proposed design can be explored, aiming for the best kind of RAM buffer for FFT architectures.

The best FFT architecture proposed in this Thesis was the SDF architecture. However, other architectures such as the SC [35] FFT may benefit from implementing the RAM buffer proposed in this Thesis.

Appendix A

Budget

The total cost of this Master Thesis is shown in table A.1. As software licenses has an annual payment period, it is supposed that it will be amortized in a period of 12 months. In the case of the Cadence EDA, it is paid in one unique payment as it is an academic license.

APPENDIX A. BUDGET

LABOUR COSTS		Hours	Cost/hour	TOTAL
Junior ASIC design Engineer		300	15,00 €	4.500 €
HARDWARE COSTS	Cost (€)	Use (months)	Amort. (years)	TOTAL
Remote server	1.500,00	6	10	75,00 €
Other equipment				0,00 €
SOFTWARE COSTS	Cost (€)	Use (months)	Amort. (years)	TOTAL
Cadence EDA	2.130,00	-	-	2.130,00 €
MATLAB	800,00	6	5	80 €
Synopsys: Front End & Verification	1.105,00	6	1	552,50 €
Europpractice membership & software	1.100,00	6	1	550,00 €
SUBTOTAL DIRECT COSTS				7.887,50 €
INDIRECT COSTS	15%	of DC		1.183,13 €
INDUSTRIAL PROFIT	6%	of DC+IC		544,23 €
SUBTOTAL				9.614,86 €
IVA	21%			2.019,12 €
TOTAL BUDGET				11.633,98 €

Table A.1: Budget.

Appendix B

Ethical, social, economic and environmental aspects

This annex presents relevant topics, related with ethical, economic, social and environmental aspects, that must be discussed. Their relationship with the project done has to be considered.

B.1 Ethical and social impacts

As mentioned in chapter 1, current 5G technologies are not yet mature enough for achieving 6G communications. In order to achieve it in the upcoming years, new architectures with better KPIs (latency, throughput and power consumption) must be released.

These 6G communications might sound unnecessarily fast for current applications. After all, why someone would like to download 10 movies per hour? In fact, current applications do not require such a speed. However, it is a must for future applications such as connected autonomous vehicles (CAV) [4] or telesurgery [5] among others.

As seen, 6G communications will be key for letting researchers in other fields continue improving the world.

Additionally, the many applications that use the FFT may take advantage of low power architectures like the ones presented here. This would impact positively on battery-powered or self-powered applications.

B.2 Economic impact

Reducing the power consumption in telecommunications and achieving 6G standards would allow, as exposed on section B.1, the introduction of future technologies.

The impact of these upcoming technologies in the world's economy is expected to come in the following years. A good example of this, are CAV vehicles [36] which already are changing the motor sector. On the one hand, there is a concern that CAV vehicles may bring bad consequences such as job-loses among driver jobs or security problems. On the other hand, CAV vehicles will improve the way we move reducing the cost of transports and allowing to use our commute time for other purposes. Recently, there have been many different actions to mitigate negative effects of CAV vehicles while improving the good ones from both governments and industries.

B.3 Environmental impact

On recent years, it has being a rise of environmental and sustainability awareness. For example, concepts like “carbon footprint” are now starting to be related with electronics, computation and telecommunication sectors [37].

A good example of this communications and computation awareness is the ban from companies such as Tesla to the usage of block-chain technology cryptocurrencies as payment of its vehicles, as it is known that the usage of Bitcoin and other cryptocurrencies have a big carbon footprint [38].

Newer computation and telecommunication techniques can help reducing carbon footprint of upcoming technologies allowing its usage without harming the environment.

Bibliography

- [1] R. Andersson and M. Garrido. Using rotator transformations to simplify FFT hardware architectures. *IEEE Trans. Circuits Syst. I*, 67(12):4784–4793, July 2020.
- [2] 3GPP TS 38.211 - Physical channels and modulation V16.3.0, September 2020. <https://www.3gpp.org/DynaReport/38-series.htm>.
- [3] S. M. Tondare, S. D. Panchal, and D. T. Kushnure. Evolutionary steps from 1G to 4.5G. *Int. J. Adv. Research Comput. Comm. Eng.*, 3(4):6163–6166, April 2014.
- [4] J. He, K. Yang, and H.-H. Chen. 6G cellular networks and connected autonomous vehicles. *IEEE Network*, pages 1–7, Nov 2020. Early Access.
- [5] S. Nayak and R. Patgiri. 6G communication technology: A vision on intelligent healthcare. In Ripon Patgiri, Anupam Biswas, and Pinki Roy, editors, *Health Informatics: A Computational Perspective in Healthcare*, pages 1–18. Springer Singapore, January 2021.
- [6] Ari Pouttu, Frank Burkhardt, Cristian Patachia, Luciano Mendes, Susanna Pirttikangas, Emery Jou, Pasi Kuvaja, Marjo Heikkilä, Tero Päivärinta, Abhishek Thakur, Mika Mäntylä, Simon Watts, Heidi Himmanen, Lauri Lovén, and Harilaos Koumaras. 6G white paper on validation and trials for verticals towards 2030’s. Technical Report 4, University of Oulu, June 2020.
- [7] Petri Ahokangas, Muhammad-Zeeshan Asghar, Fan Chen, Netta Iivari, Marcos Katz, Atte Kinnula, Josef Noll, Harri Oinas-Kukkonen, Ian Oppermann, Ella Peltonen, Hanna Saarela, Harri Saarnisaari, Anna Suorsa, Gustav Wikström, and Volker Ziegler. White paper on 6G drivers and the UN SDGs. Technical Report 2, University of Oulu, June 2020.
- [8] M. Garrido, F. Qureshi, J. Takala, and O. Gustafsson. Hardware architectures for the fast Fourier transform. In S. S. Bhattacharyya, E. F. Deprettere, R. Leupers, and J. Takala, editors, *Handbook of Signal Processing Systems*. Springer, third edition, 2019.
- [9] Europractice official website. <https://europractice-ic.com/>.

BIBLIOGRAPHY

- [10] M. Garrido. A survey on pipelined FFT hardware architectures. *J. Signal Process. Syst.*, 2021. Accepted for publication.
- [11] J. W. Cooley and J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Math. Comput.*, 19(90):297–301, April 1965.
- [12] H. Lee and I. Park. Balanced binary-tree decomposition for area-efficient pipelined FFT processing. *IEEE Trans. Circuits Syst. I*, 54(4):889–900, April 2007.
- [13] Fahad Qureshi and Oscar Gustafsson. Generation of all radix-2 fast Fourier transform algorithms using binary trees. In *Proc. European Conf. Circuit Theory Design*, pages 677–680, August 2011.
- [14] Mario Garrido. A new representation of FFT algorithms using triangular matrices. *IEEE Trans. Circuits Syst. I*, 63(10):1737–1745, October 2016.
- [15] H.L. Groginsky and G.A. Works. A pipeline fast Fourier transform. *IEEE Trans. Comput.*, C-19(11):1015–1019, October 1970.
- [16] C. Ingemarsson and O. Gustafsson. SFF—The single-stream FPGA-optimized feedforward FFT hardware architecture. *J. Signal Process. Syst.*, 90:1583–1592, November 2018.
- [17] T. Ahmed, M. Garrido, and O. Gustafsson. A 512-point 8-parallel pipelined feedforward FFT for WPAN. In *Proc. Asilomar Conf. Signals Syst. Comput.*, pages 981–984, November 2011.
- [18] M. Garrido, O. Gustafsson, and J. Grajal. Accurate rotations based on coefficient scaling. *IEEE Trans. Circuits Syst. II*, 58(10):662–666, October 2011.
- [19] Mario Garrido, F. Qureshi, and O. Gustafsson. Low-complexity multiplierless constant rotators based on combined coefficient selection and shift-and-add implementation (CCSSI). *IEEE Trans. Circuits Syst. I*, 61(7):2002–2012, July 2014.
- [20] Chia-Hsiang Yang, Tsung-Han Yu, and D. Markovic. Power and area minimization of reconfigurable FFT processors: A 3GPP-LTE example. *IEEE J. Solid-State Circuits*, 47(3):757–768, March 2012.
- [21] Wei Han, Ahmet T. Erdogan, Tughrul Arslan, and Mohd. Hasan. High-performance low-power FFT cores. *ETRI J.*, 30(3):451–460, June 2008.
- [22] Yevgen Voronenko and Markus Püschel. Multiplierless multiple constant multiplication. *ACM Trans. Algorithms*, 3:1–39, May 2007.
- [23] O. Gustafsson. A difference based adder graph heuristic for multiple constant multiplication problems. In *Proc. IEEE Int. Symp. Circuits Syst.*, pages 1097–1100, May 2007.

-
- [24] A. G. Dempster and M. D. Macleod. Multiplication by two integers using the minimum number of adders. In *Proc. IEEE Int. Symp. Circuits Syst.*, volume 2, pages 1814–1817, May 2005.
 - [25] M. Kumm, M. Hardieck, and P. Zipf. Optimization of constant matrix multiplication with low power and high throughput. *IEEE Trans. Comput.*, 66(12):2072–2080, December 2017.
 - [26] O. Gustafsson and F. Qureshi. Addition aware quantization for low complexity and high precision constant multiplication. *IEEE Signal Process. Lett.*, 17(2):173–176, February 2010.
 - [27] Jack E. Volder. The CORDIC trigonometric computing technique. *IRE Trans. Electronic Computing*, EC-8:330–334, September 1959.
 - [28] M. Garrido, P. Källström, M. Kumm, and O. Gustafsson. CORDIC II: A new improved CORDIC algorithm. *IEEE Trans. Circuits Syst. II*, 63(2):186–190, February 2016.
 - [29] Daniel Medina. Design and FPGA implementation of CORDIC architectures based on the use of uniformly distributed angles. Master’s thesis, Universidad Politécnica de Madrid, July 2019.
 - [30] S. Aggarwal, P. K. Meher, and K. Khare. Area-time efficient scaling-free CORDIC using generalized micro-rotation selection. *IEEE Trans. VLSI Syst.*, 20(8):1542–1546, August 2012.
 - [31] R. Shukla and K.C. Ray. Low latency hybrid CORDIC algorithm. *IEEE Trans. Comput.*, 63(12):3066–3078, Dec 2014.
 - [32] Mario Garrido, Rikard Andersson, Fahad Qureshi, and Oscar Gustafsson. Multiplierless unity-gain SDF FFTs. *IEEE Trans. VLSI Syst.*, 24(9):3003–3007, September 2016.
 - [33] Trio Adiono, Muh Syafiq Irsyadi, Yan Syafri Hidayat, and Ade Irawan. 64-point fast efficient FFT architecture using radix-2³ single path delay feedback. In *Proc. Int. Conf. Electrical Eng. Informatics*, volume 2, pages 654–658, August 2009.
 - [34] T. Lenart and V. Öwall. Architectures for dynamic data scaling in 2/4/8K pipeline FFT cores. *IEEE Trans. VLSI Syst.*, 14(11):1286–1290, November 2006.
 - [35] Mario Garrido, Shen-Jui Huang, Sau-Gee Chen, and Oscar Gustafsson. The serial commutator (SC) FFT. *IEEE Trans. Circuits Syst. II*, 63(10):974–978, October 2016.
 - [36] Mark Ryan. The future of transportation: Ethical, legal, social and economic impacts of self-driving vehicles in the year 2025. *Sci. Eng. Ethics*, 26(3):1185–1208, September 2020.

BIBLIOGRAPHY

- [37] Wessam M. Salama and Moustafa H. Aly. Sustainable energy in telecommunications and IT industries: Principles and solutions. In *IOP Conf. Series: Materials Sci. Eng.*, volume 1051, pages 1–10, October 2021.
- [38] Kateryna Dek and Anton Kononova. Bitcoin mining electricity consumption and carbon footprint. *Rivista di Studi sulla Sostenibilit *, pages 73–88, January 2019.